

Contents			
1 Extra	1		
1.1 Vim config	1		
2 Data Structures	1		
2.1 BIT (Fenwick Tree)	1		
2.2 BIT (Fenwick Tree) 2D	1		
2.3 DSU - Disjoint Set Union	1		
2.4 DSU - Binary Tree	1		
2.5 MinQueue	2		
2.6 Mo's Algorithm	2		
2.7 Mo with updates	2		
2.8 Segment Tree	2		
2.9 Sparse Table RMQ	3		
3 Graphs	3		
3.1 BFS 0-1	3		
3.2 Block Cut Tree (BCT)	4		
3.3 Bridges and Articulation points	4		
3.4 Dijkstra	4		
3.5 Dinic - Flow/matchings	4		
3.6 Floyd-Warshall	5		
3.7 Hopcroft-Karp - Bipartite Matching	5		
3.8 Hungarian	5		
3.9 Kosaraju - SCCs	6		
3.10 Kuhn - Bipartite Matching	6		
3.11 Min cost flow	6		
3.12 MST - Kruskal	7		
3.13 MST - Prim	7		
3.14 SCC compressing	7		
4 DP	7		
4.1 Bin Packing	7		
4.2 Broken Profile DP	8		
4.3 Convex Hull Trick (CHT)	8		
4.4 Edit Distance (Levenshtein)	8		
4.5 Knapsack - 1D	9		
4.6 Knapsack - 2D	9		
4.7 LCS - Longest Common Subsequence	9		
4.8 LiChao Tree	9		
4.9 LIS - Longest Increasing Subsequence	9		
4.10 SOSDP	10		
4.11 Subset Sum	10		
5 Trees	10		
5.1 Sum of distances	10		
5.2 Edge HLD	10		
5.3 HLD - Heavy light decomposition	11		
5.4 LCA - RMQ	11		
5.5 LCA - binary lifting	12		
6 Problemas clássicos	12		
6.1 2SAT	12		
6.2 Next Greater Element	12		
7 Strings	12		
7.1 Hashing	12		
7.2 KMP	13		
7.3 Suffix Array	13		
7.4 Suffix Automaton	13		
7.5 Z	14		
8 Math	14		
8.1 Combinatorics (Pascal's Triangle)	14		
8.1.1 Combinatorial Analysis	14		
8.2 Convolutions	16		
8.2.1 AND convolution	16		
8.2.2 GCD convolution	16		
8.2.3 LCM convolution	16		
8.2.4 OR convolution	16		
8.2.5 XOR convolution	16		
8.3 Extended Euclid	16		
8.4 Factorization	17		
8.5 FFT - Fast Fourier Transform	17		
8.6 Inclusion-Exclusion Principle	17		
8.7 Legendre's formula	17		
8.8 Linear Sieve + Möbius Function	17		
8.8.1 Applications	17		
8.9 Matrix template	18		
8.10 Mint	18		
8.11 Modular Inverse	18		
8.12 Number Theoretic Transform (NTT)	18		
8.12.1 NTT-Friendly Primes and Roots	18		
8.13 Euler's Totient	19		
9 Geometry	19		
9.1 Convex hull - Graham Scan	19		
9.2 Basic elements - geometry lib	19		
9.2.1 Polygon Area	20		
9.2.2 Point in polygon	20		
1 Extra	1		
1.1 Vim config	1		
Put these into you <code>/.vimrc</code> file. Specially, this <code>.vimrc</code> allows you to compile and run test cases using <code><leader>r</code> (<code>␣</code>)			
<pre>1 set nu ai si cindent et ts=4 sw=4 so=10 undofile cmdheight=1 2 syntax on 3 inoremap {} {}<left><return><up><end><return> 4 au BufReadPost * if line("'\"")>0 && line("'\"")<=line("\$") exe "normal! g'\"" endif 5 nnoremap <leader>r :w<CR>:let f=input('file: ')<Bar>exe 'botright vert term sh -c "g++ -fsanitize=address -g '%%. ' && ./a.out'.(empty(f)?': ' <'f).'"><CR></pre>			
2 Data Structures	14		
2.1 BIT (Fenwick Tree)	14		
Data structure that can handle point/range update and point/range queries for invertible operations. Obs: If the operation is not easily invertible (e.g. min,max,gcd), it can only answer prefix queries			
<pre>1 // [l,r] interval convention and 0-indexed parameters 2 struct BIT { 3 vector<ll> bit; 4 int n; 5 BIT(int _n = 0){ 6 n = _n; 7 bit.assign(n+1,0ll); 8 } 9 ll presum(int i){ 10 ll ans = 0; 11 for (++i; i > 0; i -= i&-i) 12 ans += bit[i]; 13 return ans; 14 } 15 ll sum(int l, int r){ 16 return presum(r) - presum(l-1); 17 } 18 void add(int i, ll v){ 19 for (++i; i <= n; i += i&-i) 20 bit[i] += v; 21 } 22 }; 23 24 struct RangeBIT { 25 BIT b1, b2; 26 int n; 27 RangeBIT(int _n){ 28 n = _n; 29 b1 = b2 = BIT(n+2); 30 } 31 void add(int l, int r, ll v){ 32 b1.add(l,v); 33 b1.add(r+1,-v); 34 b2.add(l,v*(l-1)); 35 b2.add(r+1,-v*r); 36 }</pre>			

```

37 ll presum(int i){
38     return b1.presum(i)*i - b2.presum(i);
39 }
40 ll sum(int l, int r){
41     return presum(r) - presum(l-1);
42 }
43 };

```

2.2 BIT (Fenwick Tree) 2D

2D Sum BIT, update and sum. The struct encapsulates the 1-based indexing, therefore parameters are 0-based indexed.

Query/update time: $\mathcal{O}(\log n \cdot \log m)$

Construction time (using proper constructor): $\mathcal{O}(nm)$

Space: $\mathcal{O}(nm)$

```

1 // [l,r] interval convention and 0-based indexing
2 struct BIT2D {
3     vector<vector<ll>> bit;
4     int n, m;
5     BIT2D(int _n, int _m){
6         n = _n; m = _m;
7         bit.assign(n+1, vector<ll>(m+1));
8     }
9     BIT2D(vector<vector<ll>> &a) {
10         n = a.size();
11         m = a[0].size();
12         bit.assign(n+1, vector<ll>(m+1));
13     }
14     for (int i = 1; i <= n; i++)
15         for (int j = 1; j <= m; j++)
16             bit[i][j] = a[i-1][j-1];
17
18     for (int i = 1; i <= n; i++) {
19         for (int j = 1; j <= m; j++) {
20             int nxt = j + (j&-j);
21             if (nxt <= m) bit[i][nxt] += bit[i][j];
22         }
23     }
24     for (int i = 1; i <= n; i++) {
25         int nxt = i + (i&-i);
26         if (nxt <= n) {
27             for (int j = 1; j <= m; j++)
28                 bit[nxt][j] += bit[i][j];
29         }
30     }
31 }
32 void add(int si, int sj, ll v){
33     for(int i = si+1; i <= n; i+=i&-i)
34         for (int j = sj+1; j <= m; j+=j&-j)
35             bit[i][j] += v;
36 }
37 ll presum(int si, int sj){
38     ll ans = 0;
39     for (int i = si+1; i > 0; i-=i&-i)
40         for (int j = sj+1; j > 0; j-=j&-j)
41             ans += bit[i][j];
42     return ans;
43 }
44 ll sum(ii ab, ii cd){
45     auto [a,b] = ab;
46     auto [c,d] = cd;

```

```

47     return presum(c,d) - presum(a-1,d) - presum(c,b-1)
48         + presum(a-1,b-1);
49 };

```

2.3 DSU - Disjoint Set Union

Query/update time: $\mathcal{O}(1)$

Construction time: $\mathcal{O}(n)$

Space: $\mathcal{O}(n)$

```

1 struct DSU {
2     vi p, sz;
3     DSU(int n) {
4         p.resize(n);
5         iota(p.begin(), p.end(), 0);
6         sz.assign(n, 1);
7     }
8     int find(int i) {
9         if (p[i] == i) return i;
10        return p[i] = find(p[i]);
11    }
12    bool join(int u, int v) {
13        u = find(u);
14        v = find(v);
15        if (u == v) return false;
16        if (sz[u] < sz[v]) swap(u, v);
17        p[v] = u;
18        sz[u] += sz[v];
19        return true;
20    }
21 };

```

2.4 DSU - Binary Tree

Specific code to find maximum path sums between pairs of vertices. Uses Kruskal-style MST. Query/update time: possibly $\mathcal{O}(n)$ Construction time: $\mathcal{O}(n)$ Space: $\mathcal{O}(n)$

```

1 vi d;
2 vi_ii e;
3 vi ans;
4
5 int merged;
6 vi _p, _leaf, _wei;
7 vvi adj;
8 int _find(int u) { return _p[u] == u ? u : _p[u] = _find(_p[u]); }
9 void _union(int u, int v, int w){
10     u = _find(u);
11     v = _find(v);
12     int merge_ind = merged++;
13     _p[u] = merge_ind;
14     _p[v] = merge_ind;
15     _leaf[merge_ind] = _leaf[u] + _leaf[v];
16     _wei[merge_ind] = max(_wei[u], _wei[v]);
17     adj[u].push_back(merge_ind);
18     adj[merge_ind].push_back(u);
19     adj[v].push_back(merge_ind);
20     adj[merge_ind].push_back(v);
21     merged++;
22 }

```

```

23 void make(){
24     _p = vi(2*n);
25     for(int i = 0; i < 2*n; i++) _p[i] = i;
26     _leaf = vi(2*n, 1);
27     _wei = vi(2*n);
28     for(int i = 0; i < n; i++) _wei[i] = d[i];
29     merged = 0;
30     adj = vvi(2*n);
31 }
32
33 void dfs(int u, int p){
34     for(auto &v: adj[u]){
35         if(v == p) continue;
36         ans[v] = ans[u] + (_leaf[u] - _leaf[v])*_wei[u];
37         dfs(v, u);
38     }
39 }

```

2.5 MinQueue

Minimum (or maximum) queue. All operations are $\mathcal{O}(1)$ on average. Useful for fixed length max/min queries

```

1 struct MinQueue{
2     deque<ii> q;
3     int added = 0;
4     int removed = 0;
5
6     // returns [value, index]
7     ii getmin(){ return q.front(); }
8
9     void push(int x){
10         while (!q.empty() && q.back().first > x)
11             q.pop_back();
12         q.push_back({x, added});
13         added++;
14     }
15
16     void pop(){
17         if (!q.empty() && q.front().second == removed)
18             q.pop_front();
19         removed++;
20     }
21 };

```

2.6 Mo's Algorithm

A technique for solving offline range queries on static arrays by sorting queries to minimize total pointer movement. It processes intervals by incrementally updating the range via add and remove operations. With the optimal block size, the time complexity is $\mathcal{O}((N+Q)\sqrt{N})$ or $\mathcal{O}(N\sqrt{Q})$, depending on block size choice ($\sqrt{N}$ or $N/\sqrt{Q}$). This example solves queries for distinct elements in range

```

1 struct Mo {
2     struct Query {
3         int l, r, idx, b;
4         bool operator<(const Query& o) const {
5             return b != o.b ? b < o.b :
6                 (b & 1 ? r > o.r : r < o.r);

```

```

7     }
8 };
9
10 int n, block_sz;
11
12 // custom stuff
13 vi freq, a;
14 int ans = 0;
15
16 vector<Query> queries;
17 Mo(int n) : n(n), block_sz(round(sqrt(n))) {}
18
19 // [l,r] indexed
20 void add_query(int l, int r, int i) {
21     queries.push_back({l,r,i,l/block_sz});
22 }
23 void add(int i) {
24     // add val at i
25     freq[a[i]]++;
26     if (freq[a[i]] == 1) ans++;
27 }
28 void remove(int i) {
29     // remove value at i
30     freq[a[i]]--;
31     if (freq[a[i]] == 0) ans--;
32 }
33 int get_ans() {
34     // compute current answer
35     return ans;
36 }
37
38 vi run() {
39     vi ans(queries.size());
40     sort(queries.begin(), queries.end());
41     int l = 0, r = -1;
42     for (auto& q : queries) {
43         while (l > q.l) add(--l);
44         while (r < q.r) add(++r);
45         while (l < q.l) remove(l++);
46         while (r > q.r) remove(r--);
47         ans[q.idx] = get_ans();
48     }
49     return ans;
50 }
51 };

```

2.7 Mo with updates

Also sorts queries to minimize total pointer movement, but now there is an added dimension for time (POINT updates). $\mathcal{O}(QN^{2/3})$

```

1 struct MoUpdate {
2     int n;
3     const int block_sz;
4     struct Query {
5         int l,r,t,id;
6     };
7     vector<Query> queries;
8     vector<ii> updates;
9     vector<int> a;
10
11     MoUpdate(vector<int> &v, int q) : block_sz(round(cbrt
12         (2*v.size()*v.size()))) {
13         n = v.size() + q;
14         a = v;
15         init();

```

```

15     }
16     void add_query(int l, int r) {
17         queries.push_back({l,r,(int)updates.size(),(int)
18             queries.size()});
19     }
20     void add_update(int i, int x) {
21         updates.push_back({i,x});
22     }
23     void update(int l, int r, int t) {
24         auto &[i,v] = updates[t];
25         if (l <= i && i <= r) remove(i);
26         swap(a[i],v);
27         if (l <= i && i <= r) add(i);
28     }
29     vector<int> run() {
30         vector<int> ans(queries.size());
31         sort(queries.begin(), queries.end(), [&](Query qa,
32             Query qb){
33             int l = qa.l/block_sz;
34             int r = qa.r/block_sz;
35             int ql = qb.l/block_sz;
36             int qr = qb.r/block_sz;
37             using iii = tuple<int,int,int>;
38             return iii(l, l&1 ? -r:r, r&1 ? qa.t:-qa.t) <
39                 iii(ql, ql&1 ? -qr:qr, qr&1 ? qb.t:-qb.t);
40         });
41         int l = 0, r = -1, t = 0;
42         for (auto &q : queries) {
43             while(t < q.t) update(l,r,t++);
44             while(t > q.t) update(l,r,--t);
45             while(l > q.l) add(--l);
46             while(r < q.r) add(++r);
47             while(l < q.l) remove(l++);
48             while(r > q.r) remove(r--);
49             ans[q.id] = get_ans();
50         }
51         return ans;
52     }
53     // customize depending on problem
54     int ans = 0;
55     void init(){
56         // initialize custom stuff
57     }
58     void remove(int i) {
59         // remove a[i] from the current ans
60     }
61     void add(int i) {
62         // add a[i] to the current ans
63     }
64     int get_ans() {}
65 };

```

2.8 Segment Tree

Segment tree with lazy propagation. Here the interval convention is $[l, r[$, with 0-based indexing. The example solves Kadane (max subarray sum) with point/range updates.

Query/update time: $\mathcal{O}(\log n)$

Construction time: $\mathcal{O}(n)$

Space: $\mathcal{O}(n)$

```

1 struct segtree {
2     struct node {
3         int seg, pre, suf, sum;
4     };
5     int size;
6     vector<node> nodes;
7     vector<bool> hasLazy;
8     vector<int> lazy;
9
10     node NEUTRAL = {0,0,0,0};
11
12     void debug(){
13         if (nodes.empty() || size == 0) {
14             cout << "[Empty Tree]\n"; return;
15         }
16
17         string indent = "..";
18         function<void(int, int, int, string)> print_dfs;
19
20         print_dfs = [&](int x, int lx, int rx, string
21             prefix) {
22             cout << prefix << " [" << lx << ", " << rx << "
23                 << "\n";
24
25             // debug node
26             node a = nodes[x];
27             cout << "{ ";
28             cout << "seg: " << a.seg << ' ';
29             cout << "pre: " << a.pre << ' ';
30             cout << "suf: " << a.suf << ' ';
31             cout << "sum: " << a.sum << ' ';
32             cout << "hasLazy: " << hasLazy[x] << ' ';
33             cout << "lazy: " << lazy[x] << ' ';
34             cout << "}";
35             cout << endl;
36
37             if (rx-lx <= 1) return;
38
39             int mx = (lx+rx)/2;
40             print_dfs(2*x+1, lx, mx, prefix + indent);
41             print_dfs(2*x+2, mx, rx, prefix + indent);
42         };
43         print_dfs(0, 0, size, "");
44     }
45
46     node single(int v){
47         return {v,v,v,v};
48     }
49
50     node merge(node a, node b){
51         return {
52             max(max(a.seg, b.seg), a.suf + b.pre),
53             max(a.pre, a.sum + b.pre),
54             max(b.suf, b.sum + a.suf),
55             a.sum+b.sum
56         };
57     }
58
59     void init (vi &a){
60         int n = a.size();
61         size = 1;
62         while (size < n) size *= 2;
63         nodes.assign(2*size-1, NEUTRAL);
64         hasLazy.assign(2*size-1, false);
65         lazy.assign(2*size-1, 0);
66         build(0,0,size,a);
67     }
68
69     void build(int x, int lx, int rx, vi &a){
70         if (rx-lx == 1){

```

```

69     if (lx < a.size()) nodes[x] = single(a[lx]);
70     return;
71 }
72 int mx = (lx+rx)/2;
73 build(2*x+1, lx, mx, a);
74 build(2*x+2, mx, rx, a);
75 nodes[x] = merge(nodes[2*x+1], nodes[2*x+2]);
76 }
77
78 void set(int i, int v, int x, int lx, int rx){
79     if (rx-lx == 1){
80         nodes[x] = single(v);
81         return;
82     }
83     int mx = (lx+rx)/2;
84     if (i < mx) set(i, v, 2*x+1, lx, mx);
85     else set(i, v, 2*x+2, mx, rx);
86     nodes[x] = merge(nodes[2*x+1], nodes[2*x+2]);
87 }
88
89 void set(int i, int v){
90     set(i, v, 0, 0, size);
91 }
92
93 void rangeUpdate(int l, int r, int v){
94     rangeUpdate(l, r, v, 0, 0, size);
95 }
96
97 void rangeUpdate(int l, int r, int v, int x, int lx,
98     int rx){
99     unlazy(x, lx, rx);
100    if (rx-lx < 1 || rx <= l || lx >= r) return;
101    if (l <= lx && rx <= r) return propagate(x, lx, rx, v);
102    ;
103    int mx = (lx+rx)/2;
104    rangeUpdate(l, r, v, 2*x+1, lx, mx);
105    rangeUpdate(l, r, v, 2*x+2, mx, rx);
106    nodes[x] = merge(nodes[2*x+1], nodes[2*x+2]);
107 }
108
109 node query(int l, int r){
110     return query(l, r, 0, 0, size);
111 }
112
113 node query(int l, int r, int x, int lx, int rx){
114     unlazy(x, lx, rx);
115     if (rx-lx < 1 || rx <= l || lx >= r) return NEUTRAL;
116     ;
117     if (l <= lx && rx <= r) return nodes[x];
118     int mx = (lx+rx)/2;
119     node left = query(l, r, 2*x+1, lx, mx);
120     node right = query(l, r, 2*x+2, mx, rx);
121     return merge(left, right);
122 }
123
124 void unlazy(int x, int lx, int rx){
125     if (hasLazy[x]){
126         propagate(x, lx, rx, lazy[x]);
127         hasLazy[x] = false;
128     }
129 }
130
131 void propagate(int x, int lx, int rx, int v){
132     nodes[x].sum = (rx-lx)*v;
133     nodes[x].seg = max((rx-lx)*v, 0ll);
134     nodes[x].pre = max((rx-lx)*v, 0ll);
135     nodes[x].suf = max((rx-lx)*v, 0ll);
136     if (rx-lx > 1){
137         lazy[2*x+1] = v;
138         lazy[2*x+2] = v;

```

```

136     hasLazy[2*x+1] = true;
137     hasLazy[2*x+2] = true;
138 }
139 }
140 };

```

2.9 Sparse Table RMQ

Sparse table for RMQ in $\mathcal{O}(1)$, used in many problems, including $\mathcal{O}(1)$ LCA (Trees) and LCP (SuffixArray) queries.

```

1 struct SparseTable {
2     vector<vector<ii>> st;
3
4     void build(const vi &a) {
5         int n = a.size();
6         int max_log = __lg(n)+1;
7         st.assign(max_log, vector<ii>(n));
8         for (int i = 0; i < n; i++) {
9             st[0][i] = {a[i], i};
10        }
11        for (int i = 1; i < max_log; i++) {
12            for (int j = 0; j + (1 << i) <= n; j++) {
13                // Combine the two halves
14                st[i][j] = f(st[i-1][j], st[i-1][j + (1 << (i
15                    -1))]);
16            }
17        }
18    }
19    // Returns min value and index in range [l, r]
20    // inclusive
21    ii min(int l, int r) {
22        int len = r - l + 1;
23        int k = __lg(len);
24        return f(st[k][l], st[k][r - (1<<k) + 1]);
25    }
26    inline ii f(ii l, ii r) { return std::min(l, r); }
27 };

```

3 Graphs

3.1 BFS 0-1

Time: $\mathcal{O}(n + m)$

```

1 vi bfs01(int s){
2     vi d(n, INF);
3     d[s] = 0;
4     deque<int> q;
5     q.push_front(s);
6     while(!q.empty()){
7         int u = q.front(); q.pop_front();
8         for (auto [w, v] : adj[u]){
9             if (d[u]+w < d[v]){
10                d[v] = d[u] + w;
11                if (w == 1) q.push_back(v);
12                else q.push_front(v);
13            }
14        }
15    }
16    return d;
17 }

```

3.2 Block Cut Tree (BCT)

A data structure that condenses an undirected graph into a bipartite graph using two types of nodes: B-nodes (bi-connected components) and C-nodes (articulation points). Edges strictly connect a B-node to a C-node if the articulation point belongs to that component. Yields exactly one tree per connected component.

Articulation points are explicitly represented as the C-nodes. Because an articulation point is a vertex that disconnects a component upon removal, it acts as a literal bridge in the BCT. Any path navigating between two distinct biconnected components in the original graph is forced to route through the shared C-node.

```

1 vvi build_bct(const vvi &adj){
2     int n = adj.size();
3     vi in(n, -1), low(n), st;
4     int timer = 0;
5     vvi bccs;
6
7     auto dfs = [&](auto &&dfs, int u, int p) -> void {
8         in[u] = low[u] = ++timer;
9         st.push_back(u);
10        if (p == -1 && adj[u].empty()){
11            bccs.push_back({u});
12            st.pop_back();
13            return;
14        }
15        for (int v : adj[u]) if (v != p) {
16            if (in[v] != -1) {
17                low[u] = min(low[u], in[v]);
18            } else {
19                dfs(dfs, v, u);
20                low[u] = min(low[u], low[v]);
21                if (low[v] >= in[u]){
22                    bccs.push_back({u});
23                    while(1){
24                        int w = st.back();
25                        st.pop_back();
26                        bccs.back().push_back(w);
27                        if (w == v) break;
28                    }
29                }
30            }
31        }
32    };
33    for (int i = 0; i < n; i++){
34        if (in[i] == -1) dfs(dfs, i, -1);
35    }
36    vvi bct(n+bccs.size());
37    for (int i = 0; i < bccs.size(); i++){
38        int b = n+i;
39        for (int u : bccs[i]){
40            bct[u].push_back(b);
41            bct[b].push_back(u);
42        }
43    }
44    return bct;
45 }

```

3.3 Bridges and Articulation points

DFS to get bridges and articulation points of a graph in $\mathcal{O}(n + m)$

```

1 vvi adj;
2 vi in, low;
3 int timer;
4 set<int> cut_points;
5 vector<ii> bridges;
6
7 void dfs_ap(int u, int p = -1) {
8     in[u] = low[u] = ++timer;
9     int ch = 0;
10    for (int v : adj[u]) {
11        if (v == p) continue;
12        if (in[v]) {
13            // Back-edge
14            low[u] = min(low[u], in[v]);
15        } else {
16            // Tree-edge
17            dfs_ap(v, u);
18            low[u] = min(low[u], low[v]);
19            if (low[v] >= in[u] && p != -1)
20                cut_points.insert(u);
21            ch++;
22        }
23    }
24    if (p == -1 && ch > 1)
25        cut_points.insert(u);
26 }
27
28 void dfs_bridges(int u, int p = -1) {
29     in[u] = low[u] = ++timer;
30     for (int v : adj[u]) {
31         if (v == p) continue;
32         if (in[v]) {
33             low[u] = min(low[u], in[v]);
34         } else {
35             dfs_bridges(v, u);
36             low[u] = min(low[u], low[v]);
37             if (low[v] > in[u])
38                 bridges.push_back({u, v});
39         }
40     }
41 }
42
43 void init(int n) {
44     timer = 0;
45     in.assign(n, 0);
46     low.assign(n, 0);
47     cut_points.clear();
48     bridges.clear();
49 }

```

3.4 Dijkstra

Time: $\mathcal{O}(m \log n)$

```

1 void dijkstra(int s){
2     int d, u, v;
3     dist = vi(n, INF);
4     dist[s] = 0;
5     priority_queue<ii, vii, greater<ii>> pq;
6     pq.emplace(0, s);
7     while(!pq.empty()){
8         auto [d, u] = pq.top(); pq.pop();

```

```

9         if (d > dist[u]) continue;
10
11        for (auto &[w, v] : adj[u]){
12            if (dist[v] > dist[u] + w){
13                dist[v] = dist[u] + w;
14                pq.emplace(dist[v], v);
15            }
16        }
17    }
18 }

```

3.5 Dinic - Flow/matchings

- **General Network:** $\mathcal{O}(VE \log U)$.
- **Unit Capacity Network:** $\mathcal{O}(\min(V^{2/3}, E^{1/2}) \cdot E)$. Often considered $\mathcal{O}(E\sqrt{V})$.
- **Bipartite Matching:** $\mathcal{O}(\min(V^{2/3}, E^{1/2}) \cdot E)$. Often considered $\mathcal{O}(E\sqrt{V})$.

```

1 struct Dinic {
2     struct Edge {
3         int u, v;
4         ll cap, flow = 0;
5         Edge(int u, int v, ll cap) : u(u), v(v), cap(cap) {}
6     };
7
8     const ll flow_inf = 1e18;
9     vector<Edge> edges;
10    vvi adj;
11    int n, m = 0;
12    int s, t;
13    vi level, ptr;
14    queue<int> q;
15
16    Dinic(int n): n(n) {
17        adj.resize(n);
18        level.resize(n);
19        ptr.resize(n);
20    }
21
22    void add_edge(int u, int v, ll cap) {
23        edges.emplace_back(u, v, cap);
24        edges.emplace_back(v, u, 0);
25        adj[u].push_back(m++);
26        adj[v].push_back(m++);
27    }
28
29    bool bfs(ll delta){
30        queue<int> q;
31        q.push(s);
32        while(!q.empty()){
33            int u = q.front(); q.pop();
34            for (int id : adj[u]){
35                auto &e = edges[id];
36                if (e.cap - e.flow < delta) continue;
37                if (level[e.v] != -1) continue;
38                level[e.v] = level[u] + 1;
39                q.push(e.v);
40            }
41        }
42        return level[t] != -1;
43    }

```

```

44
45    ll dfs(int u, ll pushed) {
46        if (pushed == 0) return 0;
47        if (u == t) return pushed;
48        for (int &cid = ptr[u]; cid < (int)adj[u].size(); cid++){
49            int id = adj[u][cid];
50            auto &e = edges[id];
51            if (level[u] + 1 != level[e.v]) continue;
52            ll tr = dfs(e.v, min(pushed, e.cap - e.flow));
53            if (tr == 0) continue;
54            e.flow += tr;
55            edges[id^1].flow -= tr;
56            return tr;
57        }
58        return 0;
59    }
60
61    ll maxflow(int s, int t){
62        this->s = s; this->t = t;
63        ll max_c = 0;
64        for (auto &e : edges) max_c = max(max_c, e.cap);
65
66        ll delta = 1;
67        while(delta <= max_c) delta <<= 1;
68        delta >>= 1;
69
70        ll f = 0;
71        for (; delta > 0; delta >>= 1){
72            while(true){
73                fill(level.begin(), level.end(), -1);
74                level[s] = 0;
75                if (!bfs(delta)) break;
76                fill(ptr.begin(), ptr.end(), 0);
77                while(ll pushed = dfs(s, flow_inf)) f += pushed;
78            }
79        }
80        return f;
81    }
82
83    // call constructor with (n1+n2+2) beforehand (dont
84    // add edges manually)
85    // assumes pairs are 1-indexed
86    vii maxmatchings(int n1, int n2, const vii& pairs){
87        for (int i = 1; i <= n1; i++)
88            add_edge(0, i, 1);
89
90        for (int i = 1; i <= n2; i++)
91            add_edge(i+n1, n-1, 1);
92
93        for (auto &[u, v] : pairs)
94            add_edge(u, v+n1, 1);
95
96        maxflow(0, n-1);
97
98        vii matchings;
99        for (auto &e : edges){
100            if (e.u >= 1 && e.u <= n1 && e.flow == 1 && e.v >
101                n1){
102                matchings.emplace_back(e.u, e.v-n1);
103            }
104        }
105        return matchings;
106    }
107
108    vii mincut(int s, int t){
109        maxflow(s, t);
110        queue<int> q; q.push(s);
111        vector<bool> reachable(n);

```

```

111     reachable[s] = true;
112     while(!q.empty()){
113         int u = q.front(); q.pop();
114         for (auto &id : adj[u]){
115             int v = edges[id].v;
116             if (edges[id].cap - edges[id].flow > 0 && !
117                 reachable[v]) {
118                 reachable[v] = true;
119                 q.push(v);
120             }
121         }
122     }
123     vii minCutEdges;
124     for (int i = 0; i < m; i += 2) {
125         const Edge& edge = edges[i];
126         if (reachable[edge.u] && !reachable[edge.v]) {
127             minCutEdges.emplace_back(edge.u, edge.v);
128         }
129     }
130 }
131 return minCutEdges;
132 }
133 }
134 };

```

3.6 Floyd-Warshall

Time: $\mathcal{O}(n^3)$

```

1  vvi d(n, vi(n, INF));
2  void floyd_warshall(){
3      for (int k = 0; k < n; k++)
4          for (int i = 0; i < n; i++)
5              for (int j = 0; j < n; j++)
6                  d[i][j] = min(d[i][j], d[i][k]+d[k][j]);
7  }
8  }

```

3.7 Hopcroft-Karp - Bipartite Matching

Bipartite matching such as Kuhn but faster. BFS until first layer missing match, DFS for the BFS graph to find pairings. Time: $\mathcal{O}(E\sqrt{V})$

```

1  int n, m, k;
2  vvi adj;
3  vi p, dist; /* p is in matching for [0, n[ and parent
4               for [n, n+m[ */
5  int bfs(){
6      queue<int> q;
7      dist = vi(n+m, inf);
8      for(int i = 0; i < n; i++){
9          if(p[i] == -1) q.push(i), dist[i] = 0;
10     }
11     int min_dist_match = inf;
12     while(!q.empty()){
13         int u = q.front(); q.pop();
14         if(dist[u] > min_dist_match) continue;
15         for(auto v: adj[u]){
16             if(p[v] == -1) min_dist_match = dist[u];
17             else if(dist[p[v]] == inf){
18                 dist[p[v]] = dist[u] + 1;

```

```

19         q.push(p[v]);
20     }
21     }
22     }
23     return min_dist_match != inf;
24 }
25
26 int dfs(int u){
27     for(auto v: adj[u]){
28         if(p[v] == -1 || (dist[u]+1 == dist[p[v]] && dfs(p[v]))){
29             p[v] = u;
30             p[u] = v;
31             return true;
32         }
33     }
34     dist[u] = inf;
35     return false;
36 }
37
38 int hopkarp(){
39     p = vi(n+m, -1);
40     int matchings = 0;
41     while(bfs()){
42         for(int i = 0; i < n; i++){
43             if(p[i] == -1 && dfs(i)) matchings++;
44         }
45     }
46     return matchings;
47 }
48
49 void create(){
50     adj = vvi(n+m);
51     for(int i = 0; i < k; i++){
52         int u, v;
53         cin >> u >> v; u--; v--;
54         v += n;
55         adj[u].push_back(v);
56     }
57 }

```

3.8 Hungarian

Solves minimum cost assignment for n workers and m jobs. Time: $\mathcal{O}((n+m)^3)$

```

1  // cost should be (cost[worker][job])
2  pair<int,vii> hungarian(int n, int m, const vvi &cost)
3  {
4      if (n == 0) return {0, {}};
5      int N = max(n, m);
6
7      vi u(N+1), v(N+1), p(N+1), way(N+1);
8
9      const int INF = 1e9;
10     for (int i = 1; i <= n; ++i) {
11         p[0] = i;
12         int j0 = 0;
13         vi minv(N + 1, INF);
14         vector<bool> used(N + 1, false);
15
16         do {
17             used[j0] = true;
18             int i0 = p[j0], delta = INF, j1;
19             for (int j = 1; j <= N; ++j) {

```

```

21         if (!used[j]) {
22             int cur = cost[i0-1][j-1] - u[i0] - v[j];
23             if (cur < minv[j]) {
24                 minv[j] = cur;
25                 way[j] = j0;
26             }
27             if (minv[j] < delta) {
28                 delta = minv[j];
29                 j1 = j;
30             }
31         }
32     }
33     }
34     for (int j = 0; j <= N; ++j) {
35         if (used[j]) {
36             u[p[j]] += delta;
37             v[j] -= delta;
38         } else {
39             minv[j] -= delta;
40         }
41     }
42     j0 = j1;
43     } while (p[j0] != 0);
44
45     do {
46         int j1 = way[j0];
47         p[j0] = p[j1];
48         j0 = j1;
49     } while (j0);
50
51     int total_cost = 0;
52     for (int j = 1; j <= m; ++j) {
53         if (p[j] != 0) {
54             total_cost += cost[p[j] - 1][j - 1];
55         }
56     }
57
58     // {worker, job}[] 0-indexed
59     vvi matchings;
60     for (int j = 1; j <= m; ++j) {
61         if (p[j] != 0) {
62             matchings.push_back({p[j] - 1, j - 1});
63         }
64     }
65     return {total_cost, matchings};
66 }
67 }

```

3.9 Kosaraju - SCCs

Computes the strongly connected components of a graph. Also computes the reverse topological order (if it exists). Time: $\mathcal{O}(n+m)$

```

1  void dfs1(int u){
2      vis[u] = 1;
3      for (auto v : adj[u]){
4          if (!vis[v]) dfs1(v);
5      }
6
7      ts.push_back(u);
8  }
9
10 void dfs2(int u, int c){
11     scc[u] = c;
12     for (auto v : adjT[u])
13         if (!scc[v]) dfs2(v, c);

```



```

14 }
15
16 // usage
17 for (int i = 0; i < n; i++)
18     if (!vis[i]) dfs1(i);
19
20 reverse(ts.begin(), ts.end());
21
22 int c = 1;
23 for (auto u : ts)
24     if (!scc[u]) dfs2(u, c++);

```

3.10 Kuhn - Bipartite Matching

Bipartite matching. Time: $\mathcal{O}(VE)$

```

1 int matchings;
2 vi p, vis;
3 vll match;
4
5 int dfs(int u){
6     if(vis[u]) return 0;
7     vis[u] = 1;
8     for(auto v: adj[u]){
9         if(p[v] == -1 || dfs(p[v])){
10             p[v] = u;
11             return 1;
12         }
13     }
14     return 0;
15 }
16
17 void kuhn(){
18     matchings = 0;
19     p = vi(n+m, -1);
20     for(int i = 0; i < n; i++){
21         vis = vi(n, 0);
22         matchings += dfs(i);
23     }
24     for(int i = n; i < n+m; i++){
25         if(p[i] != -1) match.push_back(ii(p[i], i));
26     }
27 }
28
29 void create(){
30     adj = vvi(n+m);
31     for(int i = 0; i < k; i++){
32         int u, v;
33         cin >> u >> v; u--; v--;
34         adj[u].push_back(v+n);
35     }
36 }

```

3.11 Min cost flow

Time: $\mathcal{O}(FE \log V)$

If negative costs are needed (maximize cost), need to run SPFA once at the start, making the solution $\mathcal{O}(EV + FE \log V)$.

```

1 struct MinCostFlow {
2     struct Edge {

```

```

3         int to, capacity, rev;
4         ll cost;
5     };
6
7     int n;
8     vector<vector<Edge>> adj;
9
10    MinCostFlow(int _n) : n(_n), adj(_n) {}
11
12    void add_edge(int from, int to, int cap, ll cost){
13        adj[from].push_back({to, cap, (int)adj[to].size(),
14                               cost});
15        adj[to].push_back({from, 0, (int)adj[from].size()-1,
16                           -cost});
17    }
18
19    // 0(FE log(V))
20    lli min_cost_flow(int s, int t, int targetFlow) {
21        int flow = 0;
22        ll total_cost = 0;
23        vll dist, h(n);
24        vi pv, pe;
25
26        // needed only if negative costs exists
27        spfa(s, h, pv, pe);
28
29        while (flow < targetFlow) {
30            dijkstra(s, h, dist, pv, pe);
31
32            if (dist[t] == INF) break;
33
34            for (int i = 0; i < n; i++) {
35                if (dist[i] < INF) {
36                    h[i] += dist[i];
37                }
38            }
39
40            int f = targetFlow - flow;
41            int cur = t;
42            while (cur != s) {
43                f = min(f, adj[pv[cur]][pe[cur]].capacity);
44                cur = pv[cur];
45            }
46
47            flow += f;
48            total_cost += f * h[t];
49            cur = t;
50            while (cur != s) {
51                Edge &e = adj[pv[cur]][pe[cur]];
52                e.capacity -= f;
53                adj[e.to][e.rev].capacity += f;
54                cur = pv[cur];
55            }
56
57            return {total_cost, flow};
58        }
59
60        // needed only if negative costs exists
61        void spfa(int s, vll &dist, vi &pv, vi &pe) {
62            dist.assign(n, INF);
63            pv.assign(n, -1);
64            pe.assign(n, -1);
65            vector<bool> inq(n, false);
66            queue<int> q;
67
68            dist[s] = 0;
69            q.push(s);
70            inq[s] = true;

```

```

71        while (!q.empty()) {
72            int u = q.front(); q.pop();
73            inq[u] = false;
74            for (int i = 0; i < adj[u].size(); i++) {
75                Edge &e = adj[u][i];
76                int v = e.to;
77                if (e.capacity > 0 && dist[v] > dist[u] + e.
78                    cost) {
79                    dist[v] = dist[u] + e.cost;
80                    pv[v] = u;
81                    pe[v] = i;
82                    if (!inq[v]) {
83                        inq[v] = true;
84                        q.push(v);
85                    }
86                }
87            }
88        }
89    }
90
91    void dijkstra(int s, vll &h, vll &dist, vi &pv, vi &
92        pe) {
93        dist.assign(n, INF);
94        pv.assign(n, -1);
95        pe.assign(n, -1);
96        dist[s] = 0;
97
98        priority_queue<lli, vector<lli>, greater<lli>> pq;
99        pq.emplace(0, s);
100
101        while (!pq.empty()) {
102            auto [d, u] = pq.top(); pq.pop();
103            if (d > dist[u]) continue;
104
105            for (int i = 0; i < adj[u].size(); i++) {
106                Edge &e = adj[u][i];
107                if (e.capacity <= 0) continue;
108                int v = e.to;
109
110                ll reduced_cost = e.cost + h[u] - h[v];
111                if (dist[u] != INF && dist[v] > dist[u] +
112                    reduced_cost) {
113                    dist[v] = dist[u] + reduced_cost;
114                    pv[v] = u;
115                    pe[v] = i;
116                    pq.push({dist[v], v});
117                }
118            }
119        }
120    }
121
122    // usage
123    int nodes = 302; // amount of nodes in the network
124    MinCostFlow mcf(nodes);
125
126    for (int i = 0; i < 150; i++){
127        mcf.add_edge(0, i+1, 1, 0); // source to node
128        mcf.add_edge(i+151, nodes-1, 1, 0); // node to sink
129    }
130
131    for (int i = 0; i < n; i++){
132        int a, b, c; cin >> a >> b >> c;
133        mcf.add_edge(a, b+150, 1, -c); // edges in between (-
134        c to maximize the cost)
135    }
136
137    // final max cost is -cost
138    auto [cost, flow] = mcf.min_cost_flow(0, nodes-1, 150);

```

3.12 MST - Kruskal

Time: $\mathcal{O}(m \log m)$

```
1 vector<pair<int,ii>> edges; // [weight, (u,v)]
2 int kruskal(int n){
3     int cost = 0;
4     DSU dsu(n); // n is the numb of vertices
5     sort(edges.begin(), edges.end());
6     for (auto &[w,uv] : edges){
7         auto [u,v] = uv;
8         if (dsu.unite(u,v)) cost += w;
9     }
10    return cost;
11 }
```

3.13 MST - Prim

Time: $\mathcal{O}(m \log n)$

```
1 vvii adj, mst;
2 vi taken;
3
4 int prim(){
5     priority_queue<iii, vector<iii>, greater<iii>> pq;
6     taken[0] = 1;
7     for (auto [w,v] : adj[0]){
8         if (!taken[v]) pq.push({w, {0,v}});
9     }
10
11    int cost = 0;
12    while (!pq.empty()){
13        auto [w,pu] = pq.top(); pq.pop();
14        auto [p,u] = pu;
15        if (!taken[u]) {
16            cost += w;
17            mst[p].emplace_back(w,u);
18            mst[u].emplace_back(w,p);
19            taken[u] = 1;
20            for (auto [w,v] : adj[u]){
21                if (!taken[v]) {
22                    pq.push({w,{u,v}});
23                }
24            }
25        }
26    }
27    return cost;
28 }
```

3.14 SCC compressing

Condensing a graph into a DAG through its strongly connected components can be useful for DP

```
1 struct SCCCondenser {
2     int n, timer, scc_cnt;
3     vi in, low, scc;
4     stack<int> st;
5
6     SCCCondenser(const vvi& adj) {
7         n = adj.size();
8         in.assign(n, 0);
9         low.assign(n, 0);
10        scc.assign(n, -1);
11        timer = scc_cnt = 0;
```

```
12        for (int i = 0; i < n; ++i)
13            if (!in[i]) dfs(i, adj);
14    }
15
16    void dfs(int u, const vvi& adj) {
17        in[u] = low[u] = ++timer;
18        st.push(u);
19        for (int v : adj[u]) {
20            if (!in[v]) {
21                dfs(v, adj);
22                low[u] = min(low[u], low[v]);
23            } else if (scc[v] == -1) {
24                low[u] = min(low[u], in[v]);
25            }
26        }
27        if (low[u] == in[u]) {
28            while (true) {
29                int v = st.top(); st.pop();
30                scc[v] = scc_cnt;
31                if (u == v) break;
32            }
33            scc_cnt++;
34        }
35    }
36
37    // Returns {DAG, Aggregated Values}
38    pair<vvi, vi> build(const vvi& adj, const vi& val) {
39        vvi dag(scc_cnt);
40        vi scc_val(scc_cnt);
41        set<ii> edges;
42
43        for (int u = 0; u < n; ++u) {
44            scc_val[scc[u]] += val[u]; // Aggregate values
45            for (int v : adj[u]) {
46                if (scc[u] != scc[v]) {
47                    if (edges.count({scc[u], scc[v]})) continue;
48                    edges.insert({scc[u], scc[v]});
49                    dag[scc[u]].push_back(scc[v]);
50                }
51            }
52        }
53        return {dag, scc_val};
54    }
55 };
```

4 DP

4.1 Bin Packing

Time: $\mathcal{O}(n \cdot 2^n)$ Space: $\mathcal{O}(2^n)$

```
1 vi w(n);
2
3 vector<ii> dp(1<<n, ii(INF,0));
4 // dp[i] = for the subset i(bitmask) (A,B) is the pair
   // where
5 // A - the min number of knapsacks to store this subset
6 // B - the min size of a used knapsack
7
8 dp[0] = ii(0,INF);
9 for (int subset = 1; subset < (1<<n); subset++){
10     for (int item = 0; item < n; item++){
11         if (!((subset>>item)&1)) continue;
12         int prevsubset = subset - (1<<item);
13         ii prev = dp[prevsubset];
14
```

```
15         if (prev.second + w[item] <= x) {
16             // can fill the knapsack, fill it
17             dp[subset] = min(dp[subset], ii(prev.first, prev.
               second+w[item]));
18         } else {
19             // cant fill the knapsack, create a new one
20             dp[subset] = min(dp[subset], ii(prev.first+1, w[
               item]));
21         }
22     }
23 }
24
25 cout << dp[(1<<n)-1].first << endl;
```

4.2 Broken Profile DP

Solves the problem of counting how many ways to fill an $n \times m$ grid using 1×2 tiles. This technique can be used whenever the state dependence is only on the previous state (column). Time: $\mathcal{O}(mn2^n)$ Space: $\mathcal{O}(mn2^n)$

```
1 int dp[1002][12][1024];
2 dp[0][0][0] = 1;
3
4 for (int i = 0; i < m; i++){
5     for (int j = 0; j < n; j++){
6         for (int mask = 0; mask < (1<<n); mask++){
7             if (mask & (1<<j)){
8                 int nxt_mask = mask - (1<<j);
9                 dp[i][j+1][nxt_mask] += dp[i][j][mask];
10                dp[i][j+1][nxt_mask] %= M;
11            } else {
12                int q = mask + (1 << j);
13                dp[i][j+1][q] += dp[i][j][mask];
14                dp[i][j+1][q] %= M;
15                if (j < n-1 && (mask & (1<<(j+1)))==0){
16                    q = mask + (1 << (j+1));
17                    dp[i][j+1][q] += dp[i][j][mask];
18                    dp[i][j+1][q] %= M;
19                }
20            }
21        }
22    }
23
24    for (int p = 0; p < (1<<n); p++){
25        dp[i+1][0][p] = dp[i][n][p];
26    }
27 }
```

4.3 Convex Hull Trick (CHT)

• Recurrence form:

TODO formulas

- **Slope monotonicity:** If coefficients a_j (slopes) are inserted in strictly decreasing (or increasing) order as j grows, and

- **Query monotonicity:** Values x_i for query come in

non-decreasing (min) or increasing (max) order consistent with slope order,

• Complexity:

- Insertion + amortized query in $\mathcal{O}(1)$ per operation (pointer walk) under monotonicity.
- Non-monotonic case, generic CHT via binary search: $\mathcal{O}(\log n)$ per query.
- General alternative: Li Chao Tree for insertion-queries in arbitrary order, $\mathcal{O}(\log M)$ per operation (where M is the domain of x).

• Constraints:

- If it cannot be written in linear form, CHT does not apply.
- If there is no monotonicity of slopes or queries, consider Li Chao Tree or CHT variant with binary search.

The example below solves the dp where the recurrence is:

TODO formulas

```
1 struct CHT {
2     struct Line { // y = mx + c
3         int m, c;
4         Line(int m, int c) : m(m), c(c) {}
5         int val(int x){
6             return m*x + c;
7         }
8         int floorDiv(int num, int den) {
9             if (den < 0) num = -num, den = -den;
10            if (num >= 0) return num / den;
11            else return - ( (-num + den - 1) / den );
12        }
13        int ceilDiv(int num, int den) {
14            if (den < 0) num = -num, den = -den;
15            if (num >= 0) return (num + den - 1) / den;
16            else return - ( (-num) / den );
17        }
18        int intersect(Line l){
19            // m1x + c1 = m2x + c2
20            // x = (c2 - c1)/(m1 - m2)
21            // if slopes are increasing, use floor div
22            return ceilDiv(l.c - c, m - l.m);
23        }
24    };
25    deque<pair<Line, int>> dq;
26
27    void insert(int m, int c){
28        Line newLine(m, c);
29        if (!dq.empty() && newLine.m == dq.back().first.m)
30            {
31                // If slopes increasing, change to <=
32                if (newLine.c >= dq.back().first.c) return;
33                else dq.pop_back();
34            }
35        // if slopes increasing, change to <=
36        while (dq.size() > 1 && dq.back().second >= dq.back()
37            .first.intersect(newLine)){
```

```
37        dq.pop_back();
38    }
39    if (dq.empty()){
40        // assuming queries are positive numbers, may
41        // change to -INF or +INF if needed
42        dq.emplace_back(newLine, 0);
43        return;
44    }
45    dq.emplace_back(newLine, dq.back().first.intersect(
46        newLine));
47
48    // dont use query and queryNonMonotonicValues in the
49    // same problem
50    int query(int x){
51        while (dq.size() > 1){
52            // if slopes increasing, change to >=
53            if (dq[1].second <= x) dq.pop_front();
54            else break;
55        }
56        return dq[0].first.val(x);
57    }
58
59    int queryNonMonotonicValues(int x){
60        int l=0, r=dq.size()-1, ans=0;
61        while (l <= r) {
62            int mid = (l+r)>>1;
63            if (dq[mid].second <= x) {
64                ans = mid;
65                l = mid + 1;
66            } else {
67                r = mid - 1;
68            }
69        }
70        return dq[ans].first.val(x);
71    }
72
73    void solve(){
74        int n, c; cin >> n >> c;
75        vi h(n);
76        for (auto &x : h) cin >> x;
77
78        vi dp(n);
79        dp[0] = 0;
80        CHT cht;
81        cht.insert(-2*h[0], h[0]*h[0]);
82        for (int i = 1; i < n; i++){
83            dp[i] = cht.query(h[i]) + c + h[i]*h[i];
84            cht.insert(-2*h[i], h[i]*h[i] + dp[i]);
85        }
86        cout << dp[n-1] << endl;
87    }
```

4.4 Edit Distance (Levenshtein)

Very similar to LCS, in the sense that it considers prefixes already computed. Time: $\mathcal{O}(mn)$ Space: $\mathcal{O}(mn)$

```
1 vvi dp(n+1, vi(m+1));
2 for (int i = 0; i <= n; i++) dp[i][0] = i;
3 for (int i = 0; i <= m; i++) dp[0][i] = i;
4 for (int i = 1; i <= n; i++){
5     for (int j = 1; j <= m; j++){
6         dp[i][j] = min(
7             min(dp[i][j-1]+1, dp[i-1][j]+1),
```

```
8             dp[i-1][j-1]+(s[i-1]!=t[j-1])
9         );
10    }
11 }
```

4.5 Knapsack - 1D

The spirit here is the same as the 2D version, but here it iterates on the knapsack capacity backwards, to ensure that the value of $dp[j-w[i]]$ is not considering the item i . Time: $\mathcal{O}(nW)$ Space: $\mathcal{O}(W)$

```
1 vi dp(W+1);
2 for (int i = 0; i < n; i++){
3     for (int j = W; j >= w[i]; j--){
4         dp[j] = max(dp[j], v[i] + dp[j-w[i]]);
5     }
6 }
```

4.6 Knapsack - 2D

Time: $\mathcal{O}(nW)$ Space: $\mathcal{O}(nW)$

```
1 vvi dp(n+1, vi(W+1));
2 for (int c = 1; c <= W; c++){
3     for (int i = 1; i <= n; i++){
4         dp[i][c] = dp[i-1][w];
5         if (c-w[i-1] >= 0) {
6             dp[i][c] = max(dp[i][c], dp[i-1][c-w[i-1]] + v[i-1]);
7         }
8     }
9 }
```

4.7 LCS - Longest Common Subsequence

Subsequence generation included here. Time: $\mathcal{O}(mn)$ Space: $\mathcal{O}(mn)$

```
1 vvi dp(n+1, vi(m+1));
2 vvii p(n+1, vii(m+1));
3
4 for (int i = 1; i <= n; i++){
5     for (int j = 1; j <= m; j++){
6         if (a[i-1] == b[j-1]) {
7             dp[i][j] = dp[i-1][j-1]+1;
8             p[i][j] = {i-1, j-1};
9         } else if (dp[i][j-1] > dp[i-1][j]){
10            dp[i][j] = dp[i][j-1];
11            p[i][j] = {i, j-1};
12        } else {
13            dp[i][j] = dp[i-1][j];
14            p[i][j] = {i-1, j};
15        }
16    }
17 }
18
19 ii pos = ii(n,m);
20 stack<int> st;
21 while(pos != ii(0,0)){
22     auto [i,j] = pos;
```

```

23 if (p[i][j] == ii(i-1,j-1)) st.push(a[i-1]);
24 pos = p[i][j];
25 }
26 cout << st.size() << endl;
27 while (!st.empty()) {
28     cout << st.top() << ' ';
29     st.pop();
30 }
31 cout << endl;

```

4.8 LiChao Tree

Generalization of CHT for linear functions that do not need to be sorted. Inspired by segtree. Queries and insertions are all $\mathcal{O}(\log M)$. Where M is the size of the query interval the tree receives.

```

1 // Li Chao tree for minimum (or maximum) over domain [L, R].
2 // T should support +, *, comparisons.
3 // For integer x use eps = 0 and discrete mid+1 splitting;
4 // For floating use eps > 0 and continuous splitting without +1.
5 template<typename T>
6 struct lichao_tree {
7     // if max lichao, change to ::min()
8     static const T identity = numeric_limits<T>::max();
9
10    struct Line {
11        T m, c;
12        Line() {
13            m = 0;
14            c = identity;
15        }
16        Line(T m, T c) : m(m), c(c) {}
17        T val(T x) { return m * x + c; }
18    };
19
20    struct Node {
21        Line line;
22        Node *lc, *rc;
23        Node() : lc(0), rc(0) {}
24    };
25
26    T L, R, eps;
27    deque<Node> buffer;
28    Node* root;
29
30    Node* new_node() {
31        buffer.emplace_back();
32        return &buffer.back();
33    }
34
35    lichao_tree() {}
36
37    lichao_tree(T _L, T _R, T _eps) {
38        init(_L, _R, _eps);
39    }
40
41    void clear() {
42        buffer.clear();
43        root = nullptr;
44    }
45
46    void init(T _L, T _R, T _eps) {
47        clear();

```

```

48     L = _L;
49     R = _R;
50     eps = _eps;
51     root = new_node();
52 }
53
54 void insert(Node* &cur, T l, T r, Line line) {
55     if (!cur) {
56         cur = new_node();
57         cur->line = line;
58         return;
59     }
60
61     T mid = l + (r - l) / 2;
62     if (r - l <= eps) return;
63
64     // if max lichao, change to >
65     if (line.val(mid) < cur->line.val(mid))
66         swap(line, cur->line);
67
68     // if max lichao, change to >
69     if (line.val(l) < cur->line.val(l)) insert(cur->lc, l, mid, line);
70     else insert(cur->rc, mid + 1, r, line);
71 }
72
73 T query(Node* &cur, T l, T r, T x) {
74     if (!cur) return identity;
75
76     T mid = l + (r - l) / 2;
77     T res = cur->line.val(x);
78     if (r - l <= eps) return res;
79
80     // if max lichao, change min to max
81     if (x <= mid) return min(res, query(cur->lc, l, mid, x));
82     else return min(res, query(cur->rc, mid + 1, r, x));
83 }
84
85 void insert(T m, T c) { insert(root, L, R, Line(m, c)); }
86
87 T query(T x) { return query(root, L, R, x); }
88 };

```

4.9 LIS - Longest Increasing Subsequence

Time: $\mathcal{O}(n \log n)$

```

1 int lis(vi &a){
2     int n = a.size();
3     vi len(n+1, INF);
4     len[0] = -INF;
5     for (int i = 0; i < n; i++){
6         int l = upper_bound(len.begin(), len.end(), a[i]) - len.begin();
7         if(len[l-1] < a[i] && a[i] < len[l]) len[l] = a[i];
8     }
9
10    int ans = 0;
11    for (int i = 0; i <= n; i++){
12        if (len[i] < INF) ans = i;
13    }
14    return ans;
15 }

```

4.10 SOSDP

```

1 int k; // amount of bits
2 vi a(1<<k);
3 // sosdp
4 for (int bit = 0; bit < k; bit++){
5     for (int mask = 0; mask < (1<<k); mask++){
6         if ((1<<bit) & mask) {
7             a[mask] += a[mask ^ (1<<bit)];
8         }
9     }
10 }
11
12 // do stuff (such as multiplication for OR convolution)
13
14 // sosdp inverse
15 for (int bit = 0; bit < k; bit++){
16     for (int mask = 0; mask < (1<<k); mask++){
17         if ((1<<bit) & mask) {
18             a[mask] -= a[mask ^ (1<<bit)];
19         }
20     }
21 }

```

4.11 Subset Sum

Almost identical to Knapsack, this code contains the subset reconstruction. Time: $\mathcal{O}(nS)$ Space: $\mathcal{O}(nS)$

```

1 vvi dp(n+1,vi(sum+1));
2 vvii p(n+1,vii(sum+1));
3
4 dp[0][0] = 1;
5
6 for (int i = 1; i <= n; i++){
7     for (int s = 1; s <= sum; s++){
8         if (s-a[i-1] >= 0 && dp[i-1][s-a[i-1]]){
9             // sum is possible taking item i
10            p[i][s] = {i-1,s-a[i-1]};
11            dp[i][s] = 1;
12        } else if (dp[i-1][s]) {
13            // sum not possible taking item i
14            // but still possible with other items (<i)
15            p[i][s] = {i-1,s};
16            dp[i][s] = 1;
17        }
18    }
19 }
20
21 if (!dp[n][target]) {
22     cout << -1 << endl;
23     return;
24 }
25
26 vi subset;
27 ii pos = {n,target};
28 while(pos != ii(0,0)){
29     auto [i,s] = pos;
30     if (p[i][s].second != s) subset.push_back(a[i-1]);
31     pos = p[i][s];
32 }

```

5 Trees

5.1 Sum of distances

Given a tree, $f(u, v) :=$ distance from u to v in the tree, compute

$$\sum_{u,v} f(u, v)$$

. Time: $\mathcal{O}(n)$

```
1 vvi adj;
2 vi sum_going_down, sum_going_up, sz;
3
4 void dfs(int u, int p){
5     for (auto v : adj[u]){
6         if (v == p) continue;
7         dfs(v, u);
8         sz[u] += sz[v];
9         sum_going_down[u] += sum_going_down[v];
10    }
11    sum_going_down[u] += sz[u];
12 }
13
14 void dfs2(int u, int p, int par_ans){
15     int up_amount = sz[0] - sz[u];
16     sum_going_up[u] += par_ans + up_amount;
17     int sum = sum_going_down[u];
18     for (auto v : adj[u]){
19         if (v == p) continue;
20         int par_amount = sz[0] - sz[v];
21         dfs2(v, u, par_ans + par_amount + sum - (
22             sum_going_down[v] + sz[v]));
23    }
24 }
25
26 void solve(){
27     int n; cin >> n;
28     adj = vvi(n);
29     sum_going_down = sum_going_up = vi(n);
30     sz = vi(n, 1);
31
32     for (int i = 1; i < n; i++){
33         int a, b; cin >> a >> b;
34         a--; b--;
35         adj[a].push_back(b);
36         adj[b].push_back(a);
37     }
38     dfs(0, 0);
39     dfs2(0, 0, 0);
40
41     for (int i = 0; i < n; i++){
42         cout << sum_going_down[i] + sum_going_up[i] << '
43         ';
44     }
45     cout << endl;
```

5.2 Edge HLD

Sometimes the value is on the edges, for this few things need to change, but here is a template. Pre-computation: $\mathcal{O}(n)$ Queries: $\mathcal{O}(\log^2 n)$

```
1 struct EdgeHLD {
2     int n, timer = 0;
3     vvi adj;
4     vi parent, depth, size, heavy, head, value;
5     vi tin, tout;
6
7     segtree seg;
8
9     void init(int _n, vvi& _adj) {
10         n = _n;
11         adj = _adj;
12         value.assign(n, 0);
13         parent.assign(n, -1);
14         depth.assign(n, 0);
15         size.assign(n, 0);
16         heavy.assign(n, -1);
17         head.assign(n, 0);
18         tin.assign(n, 0);
19         tout.assign(n, 0);
20         timer = 0;
21
22         // edgeWeight[v] = weight of edge (parent[v], v),
23         // for v>0
24         // root (0) has no parent, so its value is dummy (0)
25
26         dfs1(0, 0, 0);
27         dfs2(0, 0);
28
29         vi linear(n);
30         for (int u = 0; u < n; u++){
31             linear[tin[u]] = value[u]; // position stores
32             // edge weight
33         }
34         seg.init(linear);
35     }
36
37     int dfs1(int u, int p, int w) {
38         size[u] = 1;
39         parent[u] = p;
40         value[u] = w;
41         int max_sz = 0;
42         for (auto [v, w] : adj[u]) {
43             if (v == p) continue;
44             depth[v] = depth[u] + 1;
45             int sz = dfs1(v, u, w);
46             size[u] += sz;
47             if (sz > max_sz) {
48                 max_sz = sz;
49                 heavy[u] = v;
50             }
51         }
52         return size[u];
53     }
54
55     void dfs2(int u, int h) {
56         tin[u] = timer++;
57         head[u] = h;
58         if (heavy[u] != -1)
59             dfs2(heavy[u], h);
60         for (auto [v, w] : adj[u]) {
61             if (v != parent[u] && v != heavy[u])
62                 dfs2(v, v);
63         }
64         tout[u] = timer;
65     }
66
67     // u deve ser o filho
68     void update_edge(int u, int val) {
```

```
68     seg.set(tin[u], val);
69 }
70
71 void rangeUpdate(int u, int v, int x) {
72     while (head[u] != head[v]) {
73         if (depth[head[u]] < depth[head[v]]) swap(u, v);
74         seg.rangeUpdate(tin[head[u]], tin[u] + 1, x);
75         u = parent[head[u]];
76     }
77     if (depth[u] > depth[v]) swap(u, v);
78     seg.rangeUpdate(tin[u] + 1, tin[v] + 1, x); // +1
79     // to skip LCA's edge
80 }
81
82 void update_subtree(int u, int x) {
83     // updates all edges in subtree of u (skip incoming
84     // edge to u)
85     seg.rangeUpdate(tin[u] + 1, tout[u], x);
86 }
87
88 segtree::node query(int u, int v) {
89     segtree::node res = seg.NEUTRAL;
90     while (head[u] != head[v]) {
91         if (depth[head[u]] < depth[head[v]]) swap(u, v);
92         res = seg.merge(res, seg.query(tin[head[u]], tin[u] + 1));
93         u = parent[head[u]];
94     }
95     if (depth[u] > depth[v]) swap(u, v);
96     res = seg.merge(res, seg.query(tin[u] + 1, tin[v] + 1)); // skip LCA's edge
97     return res;
98 }
99
100 segtree::node query_subtree(int u) {
101     // query all edges in subtree of u
102     return seg.query(tin[u] + 1, tout[u]);
103 }
```

5.3 HLD - Heavy light decomposition

If you need to compute a function on a path in a tree and need to support value updates on nodes, HLD is the way. Pre-computation: $\mathcal{O}(n)$ Queries: $\mathcal{O}(\log^2 n)$

OBS: this implementation uses the same segtree as this notebook, with 0-indexing and open-closed interval convention. Ideally, just change the segtree to change the computed function, the HLD struct remains the same. OBS2: this template also supports mass updates (path/subtree) and subtree queries.

```
1 struct HLD {
2     int n, timer = 0;
3     vvi adj;
4     vi parent, depth, size, heavy, head, value;
5     vi tin, tout;
6
7     segtree seg;
8
9     void init(int _n, vi& _value, vvi& _adj) {
10         n = _n;
11         adj = _adj;
```

```

12 value = _value;
13 parent.assign(n, -1);
14 depth.assign(n, 0);
15 size.assign(n, 0);
16 heavy.assign(n, -1);
17 head.assign(n, 0);
18 tin.assign(n, 0);
19 tout.assign(n, 0);
20 timer = 0;
21
22 dfs1(0);
23 dfs2(0, 0);
24
25 vi linear(n);
26 for (int u = 0; u < n; u++)
27     linear[tin[u]] = value[u];
28
29 seg.init(linear);
30 }
31
32 int dfs1(int u) {
33     size[u] = 1;
34     int max_sz = 0;
35     for (int v : adj[u]) {
36         if (v == parent[u]) continue;
37         parent[v] = u;
38         depth[v] = depth[u] + 1;
39         int sz = dfs1(v);
40         size[u] += sz;
41         if (sz > max_sz) {
42             max_sz = sz;
43             heavy[u] = v;
44         }
45     }
46     return size[u];
47 }
48
49 void dfs2(int u, int h) {
50     tin[u] = timer++;
51     head[u] = h;
52     if (heavy[u] != -1)
53         dfs2(heavy[u], h);
54     for (int v : adj[u]) {
55         if (v != parent[u] && v != heavy[u])
56             dfs2(v, v);
57     }
58     tout[u] = timer;
59 }
60
61 void update(int u, int val) {
62     seg.set(tin[u], val);
63 }
64
65 void rangeUpdate(int u, int v, int x) {
66     while (head[u] != head[v]) {
67         if (depth[head[u]] < depth[head[v]]) swap(u, v);
68         seg.rangeUpdate(tin[head[u]], tin[u] + 1, x);
69         u = parent[head[u]];
70     }
71     if (depth[u] > depth[v]) swap(u, v);
72     seg.rangeUpdate(tin[u], tin[v] + 1, x);
73 }
74
75 void update_subtree(int u, int x) {
76     seg.rangeUpdate(tin[u], tout[u], x);
77 }
78
79 segtree::node query(int u, int v) {
80     segtree::node res = seg.NEUTRAL;
81     while (head[u] != head[v]) {

```

```

82         if (depth[head[u]] < depth[head[v]])
83             swap(u, v);
84         res = seg.merge(res, seg.query(tin[head[u]], tin[
85             u]+1));
86         u = parent[head[u]];
87     }
88     if (depth[u] > depth[v]) swap(u, v);
89     res = seg.merge(res, seg.query(tin[u], tin[v]+1));
90     return res;
91 }
92 segtree::node query_subtree(int u){
93     return seg.query(tin[u], tout[u]);
94 }
95 };

```

5.4 LCA - RMQ

Pre-computation: $\mathcal{O}(n \log n)$

Queries: $\mathcal{O}(1)$

```

1 struct LCARMQ {
2     SparseTable st;
3     vi tin, dep, et_nodes, et_deps;
4
5     LCARMQ(vvi &adj){
6         int n = adj.size(), timer = 0;
7         tin = dep = vi(n);
8         et_nodes.reserve(2*n);
9         et_deps.reserve(2*n);
10
11         auto dfs = [&](auto &&dfs, int u, int p) -> void {
12             et_nodes.push_back(u);
13             et_deps.push_back(dep[u]);
14             tin[u] = timer++;
15             for (int v : adj[u]) if (v != p) {
16                 dep[v] = dep[u]+1;
17                 dfs(dfs, v, u);
18                 et_nodes.push_back(u);
19                 et_deps.push_back(dep[u]);
20             }
21             timer++;
22         };
23         dfs(dfs, 0, 0);
24
25         st.build(et_deps);
26     }
27
28     int get(int u, int v){
29         int tu = tin[u], tv = tin[v];
30         if (tu > tv) swap(tu, tv);
31         auto [val, id] = st.min(tu, tv);
32         return et_nodes[id];
33     }
34 };

```

5.5 LCA - binary lifting

Pre-computation: $\mathcal{O}(n \log n)$ Queries: $\mathcal{O}(\log n)$ OBS: just call `dfs(root)` before starting queries.

```

1 vvi adj, up;
2 vi tin, tout;
3 int timer = 0;

```

```

4 void dfs(int u, int p){
5     tin[u] = timer++;
6     for (auto v : adj[u]){
7         if (v == p) continue;
8         up[v][0] = u;
9         for (int dist = 1; dist < LOGN; dist++){
10             up[v][dist] = up[up[v][dist-1]][dist-1];
11         }
12         dfs(v);
13     }
14     tout[u] = timer++;
15 }
16
17 int isAncestor(int u, int v){
18     return tin[u] <= tin[v] && tout[v] <= tout[u];
19 }
20
21 int lca(int u, int v){
22     if (isAncestor(u, v)) return u;
23     if (isAncestor(v, u)) return v;
24     for (int dist = LOGN-1; dist >= 0; dist--){
25         if (!isAncestor(up[u][dist], v)) u = up[u][dist];
26     }
27     return up[u][0];
28 }
29 }

```

6 Problemas clássicos

6.1 2SAT

Struct for solving 2SAT problems that supports many types of boolean expressions. To add a negated literal use `u`

```

1 // para adicionar negacao usar ~u
2 // Ex: a clausula (a v !b) se traduz para add_or(a, ~b)
3 struct TwoSatSolver {
4     int n;
5     vvi adj, adjT;
6     vector<bool> vis, assignment;
7     vi topo, scc;
8
9     void build(int _n){
10         n = 2*_n;
11         adj.assign(n, vi());
12         adjT.assign(n, vi());
13     }
14
15     int get(int u){
16         if (u < 0) return 2*(~u)+1;
17         else return 2*u;
18     }
19
20     // u -> v
21     void add_impl(int u, int v){
22         u = get(u), v = get(v);
23         adj[u].push_back(v);
24         adjT[v].push_back(u);
25         adj[v^1].push_back(u^1);
26         adjT[u^1].push_back(v^1);
27     }
28
29     // u || v
30     void add_or(int u, int v){
31         add_impl(~u, v);
32     }

```

```

33 // u && v
34 void add_and(int u, int v){
35     add_or(u,u); add_or(v,v);
36 }
37
38 // u ^ v (equiv of x != v)
39 void add_xor(int u, int v){
40     add_impl(u, ~v);
41     add_impl(~u, v);
42 }
43
44 // u == v
45 void add_equals(int u, int v){
46     add_impl(u, v);
47     add_impl(v, u);
48 }
49
50 void toposort(int u){
51     vis[u] = true;
52     for (int v : adj[u])
53         if (!vis[v]) toposort(v);
54     topo.push_back(u);
55 }
56
57 void dfs(int u, int c){
58     scc[u] = c;
59     for (int v : adjT[u])
60         if (!scc[v]) dfs(v,c);
61 }
62
63 pair<bool, vector<bool>> solve(){
64     topo.clear();
65     vis.assign(n, false);
66
67     for (int i = 0; i < n; i++)
68         if (!vis[i]) toposort(i);
69
70     reverse(topo.begin(), topo.end());
71
72     scc.assign(n, 0);
73     int c = 0;
74     for (int u : topo)
75         if (!scc[u]) dfs(u,++c);
76
77     assignment.assign(n/2, false);
78     for (int i = 0; i < n; i += 2){
79         if (scc[i] == scc[i+1]) return {false, {}};
80         assignment[i/2] = scc[i] > scc[i+1];
81     }
82     return {true, assignment};
83 }
84
85 }
86 };

```

6.2 Next Greater Element

One of the classic stack applications. Easy to translate to lower, leq or geq, just change the comparator of the while.

```

1 vi next_greater_elem(n, n);
2
3 stack<ii> st;
4 for (int i = 0; i < n; i++){
5     while (!st.empty() && st.top().first < h[i]){
6         next_greater_elem[st.top().second] = i;
7         st.pop();
8     }

```

```

9     st.emplace(h[i], i);
10 }

```

7 Strings

7.1 Hashing

Creation time: $\mathcal{O}(n)$ Access time: $\mathcal{O}(1)$ Space: $\mathcal{O}(n)$

```

1 class Hashing{
2     const int mod0 = 1e9+7;
3     vi pmod0;
4     vull pmod1;
5
6     public:
7     void CalcP(int mn, int n){
8         random_device rd;
9         uniform_int_distribution<int> dist(mn+2, mod0
10             -1);
11         int p = dist(rd);
12         if(p % 2 == 0) p--;
13         pmod0 = vi(n);
14         pmod1 = vull(n);
15         pmod0[0] = pmod1[0] = 1;
16         for(int i = 1; i < n; i++){
17             pmod0[i] = (pmod0[i-1] * p) % mod0;
18             pmod1[i] = (pmod1[i-1] * p);
19         }
20     }
21
22     viull DistinctSubstrHashes(string base, int
23         offsetVal){
24         int n = base.size();
25         viull ans;
26         for(int i = 0; i < n; i++){
27             int h0 = 0;
28             ull h1 = 0;
29             for(int j = i; j < n; j++){
30                 h0 = (h0 + (base[j]-offsetVal)*pmod0[j-
31                     i]) % mod0;
32                 h1 = (h1 + (base[j]-offsetVal)*pmod1[j-
33                     i]);
34                 ans.push_back(iull(h0, h1));
35             }
36             sort(ans.begin(), ans.end());
37             auto last = unique(ans.begin(), ans.end());
38             ans.erase(last, ans.end());
39             return ans;
40         }
41     }
42
43     viull WindowHash(string data, int offsetVal, int
44         lenWindow){
45         int n = data.size();
46         int h0 = 0;
47         ull h1 = 0;
48         viull ans;
49         for(int i = 0; i < lenWindow; i++){
50             h0 = (h0 + (data[i]+offsetVal)*pmod0[i]) %
51                 mod0;
52             h1 = (h1 + (data[i]+offsetVal)*pmod1[i]);
53         }
54         ans.push_back(iull((h0*pmod0[n-1])%mod0, h1*
55             pmod1[n-1]));
56         for(int i = lenWindow; i < n; i++){
57             h0 = (h0 - (data[i-lenWindow]+offsetVal)*
58                 pmod0[i-lenWindow]) % mod0;

```

```

51         h0 = (h0 + mod0) % mod0;
52         h0 = (h0 + (data[i]+offsetVal)*pmod0[i]) %
53             mod0;
54         h1 = (h1 - (data[i-lenWindow]+offsetVal)*
55             pmod1[i-lenWindow]);
56         h1 = (h1 + (data[i]+offsetVal)*pmod1[i]);
57         ans.push_back(iull((h0*pmod0[n-1-(i-
58             lenWindow+1)])%mod0, h1*pmod1[n-1-(i-
59             lenWindow+1)]));
60     }
61     return ans;
62 };
63 };

```

7.2 KMP

```

1 vi compute_lps(const string &pat){
2     int m = pat.length();
3     vi lps(m);
4     int len = 0;
5     for (int i = 1; i < m; i++){
6         while(len > 0 && pat[i] != pat[len])
7             len = lps[len-1];
8         if (pat[i] == pat[len]) len++;
9         lps[i] = len;
10    }
11    return lps;
12 }
13
14 // find all occurrences
15 vi kmp_search(const string &txt, const string &pat){
16     int n = txt.length();
17     int m = pat.length();
18     if (m == 0) return {};
19     vi lps = compute_lps(pat);
20     vi occurrences;
21     int j = 0;
22     for (int i = 0; i < n; i++){
23         while (j > 0 && txt[i] != pat[j])
24             j = lps[j-1];
25         if (txt[i] == pat[j]) j++;
26         if (j == m) {
27             occurrences.push_back(i-m+1);
28             j = lps[j-1];
29         }
30     }
31     return occurrences;
32 }
33
34 // find all occurrences (simpler version)
35 vi kmp_search(const string &txt, const string &pat){
36     int n = txt.length(), m = pat.length();
37     vi lps = compute_lps(pat + '#' + txt);
38     vi occurrences;
39     for (int i = 0; i < n+m+1; i++){
40         if (lps[i] == pat.length())
41             occurrences.push_back(i-m*2);
42     }
43     return occurrences;
44 }
45
46 // borda sao os prefixos que tambem sao sufixos
47 vi find_borders(const string &s){
48     vi lps = compute_lps(s);
49     int i = s.length()-1;
50     vi ans;
51     while(i > 0 && lps[i] == i){
52         ans.push_back(i);
53         i = lps[i-1];
54     }
55     return ans;
56 }

```

```

53 while (lps[i] > 0){
54     ans.push_back(lps[i]);
55     i = lps[i]-1;
56 }
57 reverse(ans.begin(), ans.end());
58 return ans;
59 }

```

7.3 Suffix Array

Time: $\mathcal{O}(n \log n)$ Space: $\mathcal{O}(n)$

```

1 struct SuffixArray {
2     int sz;
3     vi suff_ind, lcp;
4     viii suffs;
5
6     void radix_sort() {
7         if (sz <= 1) return;
8         viii suffs_new(sz);
9         vi cnt(sz + 1, 0); /*rever esse tamanho*/
10
11         for (auto& item : suffs) cnt[item.first.second]++;
12         for (int i = 1; i <= sz; ++i) cnt[i] += cnt[i - 1];
13         for (int i = sz - 1; i >= 0; --i) suffs_new[--cnt[
14             suffs[i].first.second]] = suffs[i];
15
16         cnt.assign(sz + 1, 0);
17         for (auto& item : suffs_new) cnt[item.first.first
18             ]++;
19         for (int i = 1; i <= sz; ++i) cnt[i] += cnt[i - 1];
20         for (int i = sz - 1; i >= 0; --i) suffs[--cnt[
21             suffs_new[i].first.first]] = suffs_new[i];
22     }
23
24     void build_lcp(vi& a) {
25         lcp.assign(sz, 0);
26         vi rank(sz);
27         for (int i = 0; i < sz; ++i) rank[suff_ind[i]] = i;
28
29         int h = 0;
30         for (int i = 0; i < sz; ++i) {
31             if (rank[i] == sz - 1) { h = 0; continue; }
32             if (h > 0) h--;
33             int j = suff_ind[rank[i] + 1];
34             while (i + h < sz && j + h < sz && a[i + h] == a[
35                 j + h]) h++;
36             lcp[rank[i] + 1] = h;
37         }
38     }
39
40     void build(vi& a) {
41         a.push_back(0);
42         sz = a.size();
43         suffs.resize(sz);
44         suff_ind.resize(sz);
45         vi equiv(sz);
46
47         for (int i = 0; i < sz; ++i) suffs[i] = iii(ii(a[i
48             ], a[i]), i);
49         radix_sort();
50         for (int i = 1; i < sz; ++i) {
51             auto [c, ci] = suffs[i];
52             auto [p, pi] = suffs[i-1];
53             equiv[ci] = equiv[pi] + (c > p);
54         }
55         for (int suflen = 1; suflen < sz; suflen *= 2) {

```

```

53         for (int i = 0; i < sz; ++i) {
54             suffs[i] = {equiv[i], equiv[(i + suflen) % sz
55                 ]}, i};
56         }
57         radix_sort();
58         for (int i = 1; i < sz; ++i) {
59             auto [c, ci] = suffs[i];
60             auto [p, pi] = suffs[i-1];
61             equiv[ci] = equiv[pi] + (c > p);
62         }
63     }
64     for(int i = 0; i < sz; ++i) suff_ind[i] = suffs[i].
65         second;
66     build_lcp(a);
67     a.pop_back();
68     sz--;
69     suff_ind.erase(suff_ind.begin());
70     lcp.erase(lcp.begin());
71 }
72 };

```

7.4 Suffix Automaton

```

1 struct SAM {
2     struct State {
3         int len, link;
4         ll cnt = 0;
5         int first_occ=-1;
6         vi next;
7     };
8
9     static const int alphabet = 26;
10    vector<State> st;
11    int last;
12
13    SAM(string s){
14        st.push_back({0, -1, 0, -1, vi(alphabet, -1)});
15        last = 0;
16        for (int i = 0; i < s.length(); i++){
17            extend(s[i], i);
18        }
19        calc_cnt();
20    }
21
22    void extend(char c, int id){
23        int cur = st.size();
24        st.push_back({st[last].len+1, 0, 1, id, vi(alphabet, -1)
25            });
26        int p = last;
27        while (p != -1 && st[p].next[c-'a'] == -1){
28            st[p].next[c-'a'] = cur;
29            p = st[p].link;
30        }
31        if (p == -1){
32            st[cur].link = 0;
33            last = cur;
34            return;
35        }
36        int q = st[p].next[c-'a'];
37        if (st[p].len+1 == st[q].len) {
38            st[cur].link = q;
39            last = cur;
40            return;
41        }
42        int clone = st.size();

```

```

43        st.push_back({
44            st[p].len+1,
45            st[q].link,
46            0,
47            st[q].first_occ,
48            st[q].next
49        });
50        while (p != -1 && st[p].next[c-'a'] == q){
51            st[p].next[c-'a'] = clone;
52            p = st[p].link;
53        }
54        st[q].link = st[cur].link = clone;
55        last = cur;
56    }
57
58    void calc_cnt() {
59        int sz = st.size();
60        int mx = 0;
61        for (auto &s : st) mx = max(mx, s.len);
62
63        vi c(mx+1);
64        vi nodes(sz);
65
66        for (int i = 0; i < sz; i++) c[st[i].len]++;
67        for (int i = 1; i <= mx; i++) c[i] += c[i-1];
68        for (int i = 0; i < sz; i++) nodes[-c[st[i].len]]
69            = i;
70
71        for (int i = sz-1; i >= 0; i--) {
72            int u = nodes[i];
73            if (st[u].link != -1) st[st[u].link].cnt += st[
74                u].cnt;
75        }
76
77        ll count_occurrences(string t){
78            ll cur = 0;
79            for (char c : t){
80                if (st[cur].next[c-'a'] == -1) return 0;
81                cur = st[cur].next[c-'a'];
82            }
83            return st[cur].cnt;
84        }
85
86        int first_occurrence(string t){
87            int cur = 0;
88            for (char c : t){
89                if (st[cur].next[c-'a'] == -1) return -2;
90                cur = st[cur].next[c-'a'];
91            }
92            return st[cur].first_occ-t.length()+1;
93        }
94
95        ll distinct_substrings(){
96            ll ans = 0;
97            for (int i = 1; i < st.size(); i++){
98                ans += st[i].len - st[st[i].link].len;
99            }
100            return ans;
101        }
102
103        vector<ll> distinct_substrings_perlen(int n){
104            vector<ll> diff(n+2);
105            for (int i = 1; i < st.size(); i++){
106                int l = st[st[i].link].len+1;
107                int r = st[i].len;
108                diff[l]++; diff[r+1]--;
109            }
110            vector<ll> ans(n+1);
111            ans[0] = diff[0];
112            for (int i = 1; i <= n; i++){

```



```

111     ans[i] = ans[i-1]+diff[i];
112 }
113
114 return ans;
115 }
116
117 vector<ll> dp;
118 void calc_paths(int u){
119     if (dp[u] != -1) return;
120     dp[u]=1;
121     for (int c = 0; c < alphabet; c++){
122         int v = st[u].next[c];
123         if (v == -1) continue;
124         calc_paths(v);
125         dp[u] += dp[v];
126     }
127 }
128
129 string find_kth(int k){
130     dp.assign(st.size(), -1);
131     calc_paths(0);
132     int u = 0;
133     string ans = "";
134     while(k>0){
135         for (int c = 0; c < alphabet; c++){
136             int v = st[u].next[c];
137             if (v==-1) continue;
138             bool ok = false;
139             if (k <= dp[v]){
140                 ans += 'a'+c;
141                 u = v;
142                 k--;
143                 ok = true;
144                 break;
145             }
146             if (!ok) k-=dp[v];
147         }
148     }
149     return ans;
150 }
151
152 void calc_paths_with_repetitions(int u){
153     if (dp[u] != -1) return;
154     dp[u]=st[u].cnt;
155     for (int c = 0; c < alphabet; c++){
156         int v = st[u].next[c];
157         if (v==-1) continue;
158         calc_paths_with_repetitions(v);
159         dp[u] += dp[v];
160     }
161 }
162
163 string find_kth_with_repetitions(ll k){
164     dp.assign(st.size(), -1);
165     calc_paths_with_repetitions(0);
166     int u = 0;
167     string ans = "";
168     while(k>0){
169         for (int c = 0; c < alphabet; c++){
170             int v = st[u].next[c];
171             if (v==-1) continue;
172             bool ok = false;
173             if (k <= dp[v]){
174                 ans += 'a'+c;
175                 k-=st[v].cnt;
176                 u = v;
177                 ok = true;
178                 break;
179             }
180             if (!ok) k-=dp[v];

```

```

181     }
182 }
183 return ans;
184 }
185 };

```

7.5 Z

$$z[i] := \max(k) | s[0..k-1] = s[i..i+k-1]$$

Time: $\mathcal{O}(n+m)$ Space: $\mathcal{O}(n+m)$

```

1 vi compute_z(const string &s) {
2     int n = s.length();
3     vi z(n);
4     int l = 0, r = 0;
5
6     for (int i = 1; i < n; i++) {
7         if (i <= r)
8             z[i] = min(r - i + 1, z[i-l]);
9
10        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
11            z[i]++;
12        if (i + z[i] - 1 > r) {
13            l = i;
14            r = i + z[i] - 1;
15        }
16    }
17
18    return z;
19 }
20
21 vi find_occurrences(const string &txt, const string &
22     pat){
23     vi occurrences;
24     vi z = compute_z(pat + '#' + txt);
25     int n = txt.length(), m = pat.length();
26     for (int i = 0; i < n+m+1; i++){
27         if (z[i] == m) occurrences.push_back(i-m-1);
28     }
29     return occurrences;
30 }

```

8 Math

8.1 Combinatorics (Pascal's Triangle)

Computes "n choose k". Requires factorials to be pre-computed. Time: $\mathcal{O}(\log ZAP)$

8.1.1 Combinatorial Analysis

Fundamental Counting Principles

- **Permutations:** The number of ways to arrange k items from a set of n distinct items.

$$P(n, k) = \frac{n!}{(n-k)!}$$

- **Combinations (Binomial Coefficient):** The number of ways to choose k items from a set of n distinct items, regardless of order.

$$\binom{n}{k} = C(n, k) = \frac{n!}{k!(n-k)!}$$

- **Combinations with Repetition (Stars and Bars):** The number of ways to choose k items of n types, allowing repetitions. Equivalently, the number of ways to distribute k identical balls into n distinct urns.

$$\binom{k+n-1}{n-1} = \binom{k+n-1}{k}$$

Binomial Coefficient Properties and Pascal's Triangle

- **Pascal's Triangle**

$$[n=0: \binom{0}{0} \quad n=1: \binom{1}{0} \quad \binom{1}{1} \quad n=2: \binom{2}{0} \quad \binom{2}{1} \quad \binom{2}{2}]$$

- **Stifel's Relation:** Each element in Pascal's Triangle is the sum of the two elements immediately above it.

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

- **Symmetry:** Elements of a row are symmetric with respect to the center. Choosing k elements is the same as choosing the $n-k$ elements to be left behind.

$$\binom{n}{k} = \binom{n}{n-k}$$

- **Row Sum:** The sum of all elements in row n of Pascal's Triangle (where the first row is $n=0$) is equal to 2^n .

$$\sum_{k=0}^n \binom{n}{k} = 2^n$$

- **Hockey Stick Identity:** The sum of elements in a diagonal, starting at

$$\binom{r}{r}$$

and ending at

$$\binom{n}{r}$$

, is equal to the element in the next row and next column,

$$\binom{n+1}{r+1}$$

$$\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$$

- **Binomial Theorem:**

$$(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k$$

- **Vandermonde's Identity:**

$$\sum_{j=0}^k \binom{m}{j} \binom{n}{k-j} = \binom{m+n}{k}$$

The easiest way to understand the identity is through a counting problem. Imagine you have a committee with m men and n women. How many ways can you form a subcommittee of k people?

Way 1 Direct Counting

You have a total of $m+n$ people and need to choose k of them. The number of ways to do this is simply:

$$\binom{m+n}{k}$$

Way 2 Counting by Cases

We can divide the problem into cases, based on how many men (j) are chosen for the subcommittee.

Case 0: Choose 0 men and k women. The number of ways is

$$\binom{m}{0} \binom{n}{k}$$

Case 1: Choose 1 man and $k-1$ women. The number of ways is

$$\binom{m}{1} \binom{n}{k-1}$$

Case j : Choose j men and $k-j$ women. The number of ways is

$$\binom{m}{j} \binom{n}{k-j}$$

Other Important Concepts

- **Catalan Numbers:** A sequence of natural numbers that occurs in various counting problems (e.g., number of binary trees, balanced parenthesis expressions).

$$C_n = \binom{2n}{n} - \binom{2n}{n+1} = \frac{1}{n+1} \binom{2n}{n}$$

A commonly used combinatorial proof for the Catalan numbers involves counting the number of lattice (grid) paths from $(0,0)$ to (n,n) that do not cross above the diagonal $y=x$. Each such path consists of n rightward steps and n upward steps, and the Catalan number counts the number of these "Dyck paths" that never go above the diagonal.

- **Stirling Numbers of the Second Kind:** The number of ways to partition a set of n labeled objects into k non-empty unlabeled subsets. Denoted by $S(n,k)$ or

$$\{n \atop k\}$$

$$S(n,k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

The Stirling numbers of the second kind can also be computed recursively:

$$S(n,k) = k \cdot S(n-1,k) + S(n-1,k-1)$$

with the boundary conditions:

$$S(0,0) = 1; \quad S(n,0) = 0 \text{ for } n > 0; \quad S(0,k) = 0 \text{ for } k > 0$$

- **Bell Number:** The Bell number B^n counts the total number of ways to partition a set of n labeled elements into any number (from 1 up to n) of non-empty, unlabeled subsets. It can also be written as a recurrence relation

$$B^n = \sum_{k=0}^n S(n,k)$$

- **Pigeonhole Principle:** If n items are put into m boxes, with $n > m$, then at least one box must contain more than one item.

```
1 // n escolhe k
2 // linha n, coluna k no triangulo (indexadas em 0)
3 int pascal(int n, int k){
4     int num = fat[n];
5     int den = (fat[k]*fat[n-k])%ZAP;
6     return (num*expbin(den, ZAP-2))%ZAP;
7 }
```

8.2 Convolutions

8.2.1 AND convolution

$$c[k] = \sum_{i \oplus j = k} a[i] \cdot b[j]$$

```
1 vector<mint<MOD>> and_conv(vector<mint<MOD>> a, vector<
2     mint<MOD>> b){
3     int n = a.size(); // must be pow of 2
4     for (int j = 1; j < n; j <= 1) {
5         for (int i = 0; i < n; i++) {
6             if (i&j) {
7                 a[i^j] += a[i];
8                 b[i^j] += b[i];
9             }
10        }
11    }
12    for (int i = 0; i < n; i++) a[i] *= b[i];
13
14    for (int j = 1; j < n; j <= 1) {
15        for (int i = 0; i < n; i++) {
16            if (i&j) a[i^j] -= a[i];
17        }
18    }
19    return a;
20 }
```

8.2.2 GCD convolution

$$c[k] = \sum_{\gcd(i,j)=k} a[i] \cdot b[j]$$

```

1 vector<mint<MOD>> gcd_conv(vector<int> a, vector<int> b){
2     int n = (int)max(a.size(), b.size());
3     a.resize(n);
4     b.resize(n);
5     vector<mint<MOD>> c(n);
6     for (int i = 1; i < n; i++) {
7         mint<MOD> x = 0;
8         mint<MOD> y = 0;
9         for (int j = i; j < n; j += i) {
10             x += a[j];
11             y += b[j];
12         }
13         c[i] = x*y;
14     }
15     for (int i = n-1; i >= 1; i--)
16         for (int j = 2 * i; j < n; j += i)
17             c[i] -= c[j];
18     return c;
19 }
20 
```

8.2.3 LCM convolution

$$c[k] = \sum_{\text{lcm}(i,j)=k} a[i] \cdot b[j]$$

```

1 vector<mint<MOD>> lcm_conv(vector<int> a, vector<int> b){
2     int n = (int)max(a.size(), b.size());
3     a.resize(n);
4     b.resize(n);
5     vector<mint<MOD>> c(n), x(n), y(n);
6     for (int i = 1; i < n; i++) {
7         for (int j = i; j < n; j += i) {
8             x[j] += a[i];
9             y[j] += b[i];
10        }
11        c[i] = x[i]*y[i];
12    }
13    for (int i = 1; i < n; i++)
14        for (int j = 2 * i; j < n; j += i)
15            c[j] -= c[i];
16    return c;
17 }
18 
```

8.2.4 OR convolution

$$c[k] = \sum_{i|j=k} a[i] \cdot b[j]$$

```

1 vector<mint<MOD>> or_conv(vector<mint<MOD>> a, vector<
2     mint<MOD>> b){
3     int n = a.size(); // must be pow of 2
4     for (int j = 1; j < n; j <= 1) {
5         for (int i = 0; i < n; i++) {
6             if (i&j) {
7                 a[i] += a[i~j];
8                 b[i] += b[i~j];
9             }
10        }
11    }
12    }
13    }
14    }
15    }
16    }
17    }
18    }
19    }
20    }
21    }
22    }
23    }
24    }
25    }
26    }
27    }
28    }
29    }
30    }
31    }
32    }
33    }
34    }
35    }
36    }
37    }
38    }
39    }
40    }
41    }
42    }
43    }
44    }
45    }
46    }
47    }
48    }
49    }
50    }
51    }
52    }
53    }
54    }
55    }
56    }
57    }
58    }
59    }
60    }
61    }
62    }
63    }
64    }
65    }
66    }
67    }
68    }
69    }
70    }
71    }
72    }
73    }
74    }
75    }
76    }
77    }
78    }
79    }
80    }
81    }
82    }
83    }
84    }
85    }
86    }
87    }
88    }
89    }
90    }
91    }
92    }
93    }
94    }
95    }
96    }
97    }
98    }
99    }
100   }

```

```

9     }
10    }
11    }
12    for (int i = 0; i < n; i++) a[i] *= b[i];
13    }
14    for (int j = 1; j < n; j <= 1) {
15        for (int i = 0; i < n; i++) {
16            if (i&j) a[i] -= a[i~j];
17        }
18    }
19    return a;
20 }
21 
```

8.2.5 XOR convolution

$$c[k] = \sum_{i \oplus j = k} a[i] \cdot b[j]$$

```

1 void fwht(vector<mint<MOD>> &a, bool inv){
2     int n = a.size(); // must be pow of 2
3     for (int step = 1; step < n; step <= 1){
4         for (int i = 0; i < n; i += 2*step) {
5             for (int j = i; j < i+step; j++){
6                 auto u = a[j];
7                 auto v = a[j+step];
8                 a[j] = u+v;
9                 a[j+step] = u-v;
10            }
11        }
12    }
13    if (inv) for (auto &x : a) x /= n;
14 }
15
16 vector<mint<MOD>> xor_conv(vector<mint<MOD>> a, vector<
17     mint<MOD>> b){
18     int n = a.size();
19     fwht(a,0), fwht(b,0);
20     for (int i = 0; i < n; i++) a[i] *= b[i];
21     fwht(a,1);
22     return a;
23 }
24 
```

8.3 Extended Euclid

Time: $\mathcal{O}(\log n)$.

```

1 int extended_gcd(int a, int b, int &x, int &y) {
2     x = 1, y = 0;
3     int x1 = 0, y1 = 1;
4     while (b) {
5         int q = a / b;
6         tie(x, x1) = make_tuple(x1, x - q * x1);
7         tie(y, y1) = make_tuple(y1, y - q * y1);
8         tie(a, b) = make_tuple(b, a - q * b);
9     }
10    return a;
11 }

```

8.4 Factorization

Time: $\mathcal{O}(\sqrt{n})$

```

1 // OBS: tem outras variantes mais rapidas no caderno da
2 // UDESC
3 // O(sqrt(n)) fatores repetidos
4 vi fatora(int n) {
5     vi factors;
6     for (int x = 2; x * x <= n; x++) {
7         while (n % x == 0) {
8             factors.push_back(x);
9             n /= x;
10        }
11    }
12    if (n > 1) factors.push_back(n);
13    return factors;
14 }
15
16 // O(sqrt(n))
17 // Calcula a quantidade de divisores de um numero n.
18 int qtdDivisores(int n) {
19     int ans = 1;
20     for (int i = 2; i * i <= n; i += 2) {
21         int exp = 0;
22         while (n % i == 0) {
23             n /= i; exp++;
24         }
25         if (exp > 0) ans *= (exp + 1);
26         if (i == 2) i--;
27     }
28     if (n > 1) ans *= 2;
29     return ans;
30 }
31
32 // O(sqrt(n))
33 // Calcula a soma de todos os divisores de um numero n.
34 ll somaDivisores(int n) {
35     ll ans = 1;
36     for (int i = 2; i * i <= n; i += 2) {
37         if (n % i == 0) {
38             int exp = 0;
39             while (n % i == 0) {
40                 n /= i; exp++;
41             }
42             ll aux = expbin(i, exp + 1);
43             ans *= ((aux - 1) / (i - 1));
44         }
45         if (i == 2) i--;
46     }
47     if (n > 1) ans *= (n + 1);
48     return ans;
49 }
50
51 }

```

8.5 FFT - Fast Fourier Transform

Divide and conquer algorithm used for convolutions and polynomial multiplication. Vector size a is a power of 2. Time: $\mathcal{O}(n \log n)$ Space: $\mathcal{O}(n)$

```

1 void fft(vector<cd> &a, bool invert){
2     int len = a.size();
3     for(int i = 1, j = 0; i < len; i++){

```

```

4   int bit = len >> 1;
5   while(bit & j){
6       j ^= bit;
7       bit >>= 1;
8   }
9   j ^= bit;
10  if(i < j) swap(a[i], a[j]);
11  }
12  for(int l = 2; l <= len; l <= 1){
13      double ang = 2*PI/l * (invert ? -1: 1);
14      cd wd(cos(ang), sin(ang));
15      for(int i = 0; i < len; i += l){
16          cd w(1);
17          for(int j = 0; j < l/2; j++){
18              cd u = a[i+j], v = a[i+j+l/2];
19              a[i+j] = u*w*v;
20              a[i+j+l/2] = u-w*v;
21              w *= wd;
22          }
23      }
24  }
25  if(invert){
26      for(int i = 0; i < len; i++){
27          a[i] /= len;
28      }
29  }
30 }

```

8.6 Inclusion-Exclusion Principle

TODO: rewrite math statement

```

1 // Exemplo:
2 // Contar numeros de 1 a n divisiveis por uma lista de
3 // primos.
4 int n;
5 vi primes;
6 int factors = primes.size();
7 int total_divisible = 0;
8 // Itera pelas bitmasks nao vazias de 'primes'
9 for (int i = 1; i < (1 << factors); i++) {
10     int current_lcm = 1;
11     int subset_size = 0;
12
13     // calcula lcm do subconjunto
14     for (int j = 0; j < factors; j++) {
15         if (i & (1<<j)) {
16             subset_size++;
17             current_lcm = lcm(current_lcm, primes[j]);
18             if (current_lcm > n) break;
19         }
20     }
21
22     if (current_lcm > n) {
23         continue;
24     }
25
26     int count = n / current_lcm;
27
28     // Aplica o Principio da Inclusao-Exclusao:
29     // Se o tamanho do subconjunto eh impar, adiciona.
30     // Se o tamanho do subconjunto eh par, subtrai.
31     if (subset_size & 1) {
32         total_divisible += count;
33     } else {
34         total_divisible -= count;
35     }
36 }

```

```
36 }
```

8.7 Legendre's formula

Computes the largest power of a prime p in n!

```

1 int legendre(int n, int p){
2     int ans = 0;
3     while(n>0){
4         n /= p; ans += n;
5     }
6     return ans;
7 }

```

8.8 Linear Sieve + Möbius Function

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1 \\ (-1)^k & \text{if } n \text{ is a product of } k \text{ distinct primes} \\ 0 & \text{if } p^2 \mid n \text{ for some prime } p \end{cases}$$

8.8.1 Applications

- **Inclusion-Exclusion:** Naturally generates $1, -1, 0$ coefficients for prime factor sets.
- **Möbius Inversion:** Converts $g(n) = \sum_{d|n} f(d)$ to $f(n) = \sum_{d|n} \mu(d)g(\frac{n}{d})$. Often reduces $O(N)$ divisor sums to $O(\sqrt{N})$.
- **Dirichlet Convolution:** $\mu * 1 = \epsilon$. Core to sub-linear prefix sum algorithms (e.g., DuSieve) yielding $O(N^{2/3})$ time.

```

1 auto sieve = [](int n) -> pair<vi,vi> {
2     vi primes, mu(n+1);
3     mu[1] = 1;
4     vector<bool> is_prime(n+1, true);
5     is_prime[0] = is_prime[1] = false;
6     for (int i = 2; i <= n; ++i) {
7         if (is_prime[i]) {
8             primes.push_back(i);
9             mu[i] = -1;
10        }
11        for (int p : primes) {
12            if (i*p > n) break;
13            is_prime[i*p] = false;
14            if (i%p == 0) {
15                mu[i*p] = 0;
16                break;
17            }
18            mu[i*p] = -mu[i];
19        }
20    }
21    return {primes, mu};
22 };

```

8.9 Matrix template

A template for square matrices, used for solving linear recurrences with fast exponentiation, memory is on a 1D vector, but can be accessed with `A[i][j]` normally (custom `operator[]`)

```

1 template<typename T>
2 struct mat{
3     vector<T> m;
4     int n;
5
6     mat(int _n = 0, bool identity = false) : n(_n) {
7         m.resize(n*n);
8         if (!identity) return;
9         for (int i = 0; i < n; i++)
10             m[i*n+i] = 1;
11     }
12
13     mat& operator+=(const mat& o){
14         for (int i = 0; i < n; i++){
15             int ra = i*n;
16             for (int j = 0; j < n; j++){
17                 m[ra+j] += o.m[ra+j];
18             }
19         }
20         return *this;
21     }
22     mat& operator-=(const mat& o){
23         for (int i = 0; i < n; i++){
24             int ra = i*n;
25             for (int j = 0; j < n; j++){
26                 m[ra+j] -= o.m[ra+j];
27             }
28         }
29         return *this;
30     }
31     mat& operator*=(const mat& o){
32         vector<T> ans(n*n);
33         for (int i = 0; i < n; i++){
34             for (int k = 0; k < n; k++){
35                 int ra = i*n, rb = k*n;
36                 for (int j = 0; j < n; j++){
37                     ans[ra+j] += m[ra+k] * o.m[rb+j];
38                 }
39             }
40         }
41         this->m = ans;
42         return *this;
43     }
44     friend mat operator+(mat a, const mat& b){
45         return a+b;
46     }
47     friend mat operator-(mat a, const mat& b){
48         return a-b;
49     }
50     friend mat operator*(mat a, const mat& b){
51         return a*b;
52     }
53     T* operator[](int i){
54         return &m[i*n];
55     }
56     vector<T> operator*(const vector<T> &v){
57         vector<T> ans(n);
58         for (int i = 0; i < n; i++){
59             int ra = i*n;
60             for (int j = 0; j < n; j++){
61                 ans[i] += m[ra+j]*v[j];
62             }
63         }
64     }
65 }

```

```

63     return ans;
64 }
65 mat operator^(int e){
66     return exp(*this, e);
67 }
68 static mat exp(mat b, ll e){
69     mat ans = mat(b.n, true);
70     while(e > 0){
71         if(e & 1) ans *= b;
72         b *= b;
73         e /= 2;
74     }
75     return ans;
76 }
77 };

```

8.10 Mint

```

1  template<ll MOD>
2  struct mint {
3      ll val;
4      mint(ll v = 0) {
5          if (v < 0) v = v % MOD + MOD;
6          if (v >= MOD) v %= MOD;
7          val = v;
8      }
9      mint& operator+=(const mint& other) {
10         val += other.val;
11         if (val >= MOD) val -= MOD;
12         return *this;
13     }
14     mint& operator-=(const mint& other) {
15         val -= other.val;
16         if (val < 0) val += MOD;
17         return *this;
18     }
19     mint& operator*=(const mint& other) {
20         val = (val * other.val) % MOD;
21         return *this;
22     }
23     mint& operator/=(const mint& other) {
24         val = (val * inv(other).val) % MOD;
25         return *this;
26     }
27     friend mint operator+(mint a, const mint& b) {
28         return a + b;
29     }
30     friend mint operator-(mint a, const mint& b) {
31         return a - b;
32     }
33     friend mint operator*(mint a, const mint& b) {
34         return a * b;
35     }
36     friend mint operator/(mint a, const mint& b) {
37         return a / b;
38     }
39     static mint power(mint b, ll e) {
40         mint ans = 1;
41         while (e > 0) {
42             if (e & 1) ans *= b;
43             b *= b;
44             e /= 2;
45         }
46         return ans;
47     }
48     static mint inv(mint n) { return power(n, MOD - 2); }
49 };

```

8.11 Modular Inverse

If m is prime, can use binary exponentiation to compute a^{p-2} (Fermat's Little Theorem).

This code works for non-prime m , as long as it is coprime to a .

Time: $\mathcal{O}(\log m)$

```

1  int modInverse(int a, int m) {
2      int x, y;
3      int g = extendedGcd(a, m, x, y);
4      if (g != 1) return -1;
5      return (x % m + m) % m;
6  }

```

8.12 Number Theoretic Transform (NTT)

NTT is a fast algorithm (analogous to FFT) for polynomial multiplication modulo a special prime. It requires a prime modulus $p = c \cdot 2^k + 1$ (a "primitive root prime") and a primitive 2^k -th root of unity modulo p .

- **Prime Choices:** To use NTT, pick a modulus and a matching primitive root (see table below). For arbitrary moduli (e.g., $10^9 + 7$), multiply with several NTT-friendly primes and reconstruct with CRT (see `crt_multiply`).
- **Time Complexity:** $\mathcal{O}(n \log n)$ for polynomial multiplication.

8.12.1 NTT-Friendly Primes and Roots

NTT-friendly primes and their primitive roots:

- Mod: 998244353, Root: 3, Max N: 2^{23}
- Mod: 734003201, Root: 3, Max N: 2^{20}
- Mod: 167772161, Root: 3, Max N: 2^{25}
- Mod: 469762049, Root: 3, Max N: 2^{26}

Use the modulus as MOD and the root as ROOT when instantiating the NTT.

- For large/concrete moduli, see `crt_multiply` in the code for a multi-modulus solution with Chinese Remainder Theorem (CRT).

```

1  namespace NTT{
2      template<typename T, ll MOD, ll ROOT>
3      void transform(vector<T>& a, bool invert) {
4          int n = a.size();
5          for (int i = 1, j = 0; i < n; i++) {
6              int bit = n >> 1;

```

```

8              for (; j & bit; bit >>= 1) j ^= bit;
9              j ^= bit;
10             if (i < j) swap(a[i], a[j]);
11         }
12         for (int len = 2; len <= n; len <= 1) {
13             T wlen = T::power(ROOT, (MOD - 1) / len);
14             if (invert) wlen = T::inv(wlen);
15             for (int i = 0; i < n; i += len) {
16                 T w = 1;
17                 for (int j = 0; j < len / 2; j++) {
18                     T u = a[i + j], v = a[i + j + len / 2] * w;
19                     a[i + j] = u + v;
20                     a[i + j + len / 2] = u - v;
21                     w *= wlen;
22                 }
23             }
24         }
25         if (invert) {
26             T n_inv = T::inv(n);
27             for (T& x : a) x *= n_inv;
28         }
29     }
30 }
31
32 template<typename T, ll MOD, ll ROOT>
33 vector<ll> multiply(const vector<ll>& a, const
34 vector<ll>& b) {
35     vector<T> fa(a.begin(), a.end()), fb(b.begin(),
36 b.end());
37     int n = 1;
38     while (n < a.size() + b.size()) n <= 1;
39     fa.resize(n);
40     fb.resize(n);
41     transform<T, MOD, ROOT>(fa, false);
42     transform<T, MOD, ROOT>(fb, false);
43     for (int i = 0; i < n; i++) fa[i] *= fb[i];
44     transform<T, MOD, ROOT>(fa, true);
45     vector<ll> result(n);
46     for (int i = 0; i < n; i++) result[i] = fa[i].
47 val;
48     return result;
49 }
50
51 vector<ll> crt_multiply(const vector<ll>& a, const
52 vector<ll>& b) {
53     const ll mod1 = 998244353;
54     const ll root1 = 3;
55     using mint1 = mint<mod1>;
56     vector<ll> ans1 = NTT::multiply<mint1, mod1,
57 root1>(a, b);
58
59     const ll mod2 = 1004535809;
60     const ll root2 = 3;
61     using mint2 = mint<mod2>;
62     vector<ll> ans2 = NTT::multiply<mint2, mod2,
63 root2>(a, b);
64
65     int ans_size = a.size() + b.size() - 2;
66     ll M1_inv_M2 = mint<mod2>::inv(mod1).val;
67     vector<ll> final_result(ans_size + 1);
68     for (int i = 0; i <= ans_size; ++i) {
69         ll v1 = ans1[i];
70         ll v2 = ans2[i];

```

```

71         ll k = ((v2 - v1 + mod2) % mod2 * M1_inv_M2
72             ) % mod2;
73         final_result[i] = v1 + k * mod1;
74     }
75     return final_result;
76 }

```

8.13 Euler's Totient

Returns the amount of numbers smaller than n that are coprime to n . Time: $\mathcal{O}(\sqrt{n})$

```

1 int phi(int n){
2     int ans = n;
3     for (int i = 2; i*i <= n; i++){
4         if (n%i == 0){
5             while(n%i == 0) n/=i;
6             ans -= ans/i;
7         }
8     }
9     if (n>1) ans -= ans/n;
10    return ans;
11 }

```

9 Geometry

9.1 Convex hull - Graham Scan

Time: $\mathcal{O}(n \log n)$

```

1 #define CLOCKWISE -1
2 #define COUNTERCLOCKWISE 1
3 #define INCLUDE_COLLINEAR 0 // pode mudar
4
5 struct Point {
6     ll x, y;
7     bool operator==(Point const& t) const {
8         return x == t.x && y == t.y;
9     }
10 };
11
12 struct Vec {
13     int x, y, z;
14 };
15
16 Vec cross(Vec v1, Vec v2){
17     int x = v1.y*v2.z - v1.z*v2.y;
18     int y = -v1.x*v2.z + v1.z*v2.x;
19     int z = v1.x*v2.y - v1.y*v2.x;
20     return {x,y,z};
21 }
22
23 ll dist2(Point p1, Point p2){
24     int dx = p1.x-p2.x;
25     int dy = p1.y-p2.y;
26     return dx*dx+dy*dy;
27 }
28
29 ll orientation(Point pivot, Point a, Point b){
30     Vec va = {a.x-pivot.x, a.y-pivot.y, 0};
31     Vec vb = {b.x-pivot.x, b.y-pivot.y, 0};
32     Vec v = cross(va,vb);

```

```

33     if (v.z < 0) return CLOCKWISE;
34     if (v.z > 0) return COUNTERCLOCKWISE;
35     return 0;
36 }
37
38 bool clock_wise(Point pivot, Point a, Point b) {
39     int o = orientation(pivot, a, b);
40     return o < 0 || (INCLUDE_COLLINEAR && o == 0);
41 }
42
43 bool collinear(Point a, Point b, Point c) { return
44     orientation(a, b, c) == 0; }
45
46 vector<Point> convex_hull(vector<Point> &points, bool
47     counterClockwise) {
48     int n = points.size();
49     Point pivot = *min_element(points.begin(), points.
50         end(), [](Point a, Point b) {
51             return ii(a.y, a.x) < ii(b.y, b.x);
52         });
53     sort(points.begin(), points.end(), [&](Point a,
54         Point b) {
55         int o = orientation(pivot, a, b);
56         if (o == 0) return dist2(pivot, a) < dist2(
57             pivot, b);
58         return o == CLOCKWISE;
59     });
60
61     if (INCLUDE_COLLINEAR) {
62         int i = n-1;
63         while (i >= 0 && collinear(pivot, points[i],
64             points.back())) i--;
65         reverse(points.begin()+i+1, points.end());
66     }
67
68     vector<Point> hull;
69     for (auto p : points) {
70         while (hull.size() > 1 && !clock_wise(hull[hull
71             .size()-2], hull.back(), p))
72             hull.pop_back();
73         hull.push_back(p);
74     }
75
76     if (!INCLUDE_COLLINEAR && hull.size() == 2 && hull
77         [0] == hull[1])
78         hull.pop_back();
79
80     if (counterClockwise && hull.size() > 1) {
81         vector<Point> reversed_hull = hull;
82         reverse(reversed_hull.begin() + 1,
83             reversed_hull.end());
84         return reversed_hull;
85     }
86     return hull;
87 }

```

9.2 Basic elements - geometry lib

- Basic elements for using the geometry lib, contains points, vector operations and distances between points, distance between point and segment, distance between segments, segment intersection check, orientation check (ccw).
- Always use long double for floating point. Only use floating point if indispensable.

- For $a == b$, use $|a - b| < \text{eps}!!!!$

Time: $\mathcal{O}(1)$

9.2.1 Polygon Area

- Heron's Formula for triangle area:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

, where a , b , and c are the triangle sides and $s = (a + b + c)/2$

TODO Shoelace

- Pick's Theorem for polygon area with integer coordinates:

$$A = a + b/2 - 1$$

, where a is the number of integer coordinates inside the polygon and b is the number of integer coordinates on the polygon boundary. b can be calculated for each edge as

$$b = \gcd(x_i + 1 - x_i, y_i + 1 - y_i) + 1$$

Polygon Area Time: $\mathcal{O}(n)$

9.2.2 Point in polygon

Sum of edge angles relative to the point must sum to 2π
Time: $\mathcal{O}(n \log n)$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long double ld;
4 #define eps 1e-9
5 #define pi 3.141592653589
6 #define int long long int
7
8
9 struct pt {
10     int x, y;
11     int operator==(pt b) {
12         return x == b.x && y == b.y;
13     }
14     int operator<(pt b) {
15         if(x == b.x) return y < b.y;
16         return x < b.x;
17     }
18     pt operator-(pt b) {
19         return {x - b.x, y - b.y};
20     }
21     pt operator+(pt b) {
22         return {x+b.x, y + b.y};
23     }

```



```

24 };
25 int cross(pt u, pt v) {
26     return u.x * v.y - u.y * v.x;
27 }
28 int dot(pt u, pt v) {
29     return u.x * v.x + u.y * v.y;
30 }
31 ld norm(pt u) {
32     return sqrt(dot(u, u));
33 }
34 ld dist(pt u, pt v) {
35     return norm(u - v);
36 }
37 int ccw(pt u, pt v) { // cuidado com colineares!!!!
38     return (cross(u, v) > eps)?1:((fabs(cross(u, v)) <
39         eps)?0:-1);
40 }
41 int pointInSegment(pt a, pt u, pt v) { // checks if a
42     // lies in uv
43     if(ccw(v - u, a - u)) return 0;
44     vector<pt> pts = {a, u, v};
45     sort(pts.begin(), pts.end());
46     return pts[1] == a;
47 }
48 ld angle(pt u, pt v) { // angle between two vectors
49     ld c = cross(u, v);
50     ld d = dot(u, v);
51     return atan2l(c, d);

```

```

50 }
51 int intersect(pt sa, pt sb, pt ra, pt rb) { // not sure
52     // if it works when one of the segments is a point
53     pt s = sb - sa, r = rb - ra;
54     if(pointInSegment(sa, ra, rb) || pointInSegment(sb,
55         ra, rb) || pointInSegment(ra, sa, sb) ||
56         pointInSegment(rb, sa, sb)) return 1;
57     return !(ccw(s, ra - sa) == ccw(s, rb - sa) || ccw(
58         r, sa - ra) == ccw(r, sb - ra));
59 }
60 ld polygonArea(vector<pt>& p) { // not signed (for
61     // signed area remove the absolute value at the end)
62     ld area = 0;
63     int n = p.size() - 1; // p[n] = p[0]
64     for(int i = 0; i < n; i++) {
65         area += cross(p[i], p[i + 1]);
66     }
67     return fabs(area)/2;
68 }
69 int pointInPolygon(pt a, vector<pt>& p) { // returns 0
70     // for point in BOUNDARY, 1 for point in polygon and
71     // -1 for outside
72     ld total = 0;
73     int n = p.size() - 1;
74     for(int i = 0; i < n; i++) {
75         pt u = p[i] - a;
76         pt v = p[i + 1] - a;

```

```

71         if(fabs(dist(p[i], a) + dist(p[i + 1], a) -
72             dist(p[i], p[i + 1])) < eps) {
73             return 0;
74         }
75         total += angle(u, v);
76     }
77     return (fabs(fabs(total) - 2 * pi) < eps)?1:-1;
78 }
79
80 signed main() {
81     int n, m; scanf("%lld %lld", &n, &m);
82     vector<pt> p(n + 1);
83     for(int i = 0; i < n; i++) {
84         scanf("%lld %lld", &p[i].x, &p[i].y);
85     }
86     p[n] = p[0];
87     while(m--) {
88         pt a; scanf("%lld %lld", &a.x, &a.y);
89         int ans = pointInPolygon(a, p);
90         printf("%s\n", (ans > 0)?"INSIDE":(ans?"OUTSIDE"
91             ":"BOUNDARY));
92     }
93 }

```